

GRADUATION THESIS

MycoAI Retrieval: A System for Fungal Species Identification using Image Retrieval and Agentic Engineering

Prepared by: Pham Thanh Dat

MSSV: 20226024

Supervisor: Ms Nguyen Thi Oanh

Ngày 16 tháng 6 năm 2026

ACKNOWLEDGMENT

My journey into this thesis began with a question I asked myself as an ICT student who loves building products: how can computer vision solve real-world problems that matter? I was not content with toy datasets or classroom exercises—I wanted to work on something where the technology tangibly helps people.

This aspiration led me to join the MycoAI lab under the guidance of **Dr. Oanh Nguyen** from Hanoi University of Science and Technology (HUST), a researcher deeply dedicated to computer vision. Through Dr. Oanh, I was introduced to **Professor Duong Vu**, a data scientist based in the Netherlands who has spent years specializing in fungal classification and founded the MycoAI laboratory to bridge AI and mycology.

Rather than handing me a well-defined problem, they gave me something more valuable: access to a real fungal image dataset and a single open-ended question—*“What can you do to help a mycologist?”*

That question became my compass. I spent the early months not building models, but exploring the dataset, reading mycology literature, and speaking with domain experts to understand their workflow. Mycologists spend hours manually comparing colony photographs against reference books. A misidentification can delay research, misdirect treatment, or waste valuable lab resources. The bottleneck was not the microscope—it was the lookup.

I would like to express my sincere gratitude to **Dr. Oanh Nguyen** for her unwavering support throughout this project—for the guidance, the ideas, and the deep knowledge of computer vision techniques she shared with me at every stage. Her mentorship shaped not just the outcome of this thesis, but how I approach a problem as an engineer.

I am equally grateful to **Professor Duong Vu**, whose clear articulation of the project’s requirements grounded this work in real mycologist needs. His domain expertise kept the system honest: every feature and every experiment was measured against the question of whether it actually helps the person in the lab.

Hanoi, June 2025

ABSTRACT

This thesis presents **MycoAI Retrieval**, an image-based fungal species identification system that combines a retrieval-based classification pipeline with a production-ready web application and an Agentic Engineering development methodology.

The retrieval pipeline operates on 435 Petri dish images across 8 *Penicillium* species and 31 strains. Colony images are segmented using K-Means clustering with a novel Local K=2 Shrink method to handle agar flare, then embedded via fine-tuned EfficientNetB1 (83.3% strain-level accuracy). Vector search over Qdrant with weighted aggregation converts segment-level similarity into species-level predictions, with no retraining required when new species are added.

The web application wraps this pipeline into a governed, multi-role product (User / Data Owner) with 47 REST API endpoints, 14 database tables, and a dual-database architecture (PostgreSQL for governance, Qdrant for search). The system supports the complete lifecycle: upload → segment → retrieve → provide feedback → review → index → maintain model.

The development process itself was accelerated by **Agentic Engineering**: a five-agent orchestration layer (Autolab, Researcher, Planner, Worker, Reporter) that autonomously proposes hypotheses, runs experiments in isolated git worktrees, and surfaces the best-performing configurations—reducing the manual experiment cycle to an automated loop.

TABLE OF CONTENTS

ACKNOWLEDGMENT	i
ABSTRACT	ii
CHAPTER 1. INTRODUCTION.....	1
1.1 Problem Statement.....	1
1.1.1 Overview	1
1.1.2 Dataset Description	1
1.1.3 Scope of Work	8
1.1.4 Proposed Solution	9
1.1.5 Rationale.....	11
CHAPTER 2. FUNGAL SPECIES RETRIEVAL MODEL	14
2.1 Fungal Species Retrieval Model	14
2.1.1 Motivation.....	14
2.1.2 Related Work.....	14
2.1.3 Methodology	14
2.1.4 Experiments and Results.....	22
2.1.5 Discussion.....	28
CHAPTER 3. WEB APPLICATION DESIGN & IMPLEMENTATION	
31	
3.1 Web Application.....	31
3.1.1 Requirements Analysis	31
3.1.2 System Architecture	34
3.1.3 Database Design	38
3.1.4 API Design.....	41
3.1.5 Frontend Architecture.....	46
	iii

3.1.6 Core Backend Pipelines	49
3.1.7 Deployment Architecture	52
3.1.8 Data Visualization Charts	53
3.1.9 Summary	54
CHAPTER 4. AGENTIC ENGINEERING & EVALUATION.....	56
4.1 Agentic Engineering	56
4.1.1 Motivation.....	56
4.1.2 Multi-Agent Architecture	56
4.1.3 Reducing Hallucinations.....	56
4.1.4 Overcoming Context Window Limits.....	57
CHAPTER 5. CONCLUSION	58
5.1 Conclusion.....	58
5.1.1 Summary	58
5.1.2 Future Work	58
REFERENCE	60
CHAPTER A. USE CASE DESCRIPTIONS	62
A.1 Use Case Details.....	62

LIST OF FIGURES

Figure 1.1	Three fungal strains across five growth media (oblique view). Green border = same species as reference strain; red border = dif- ferent species. All images from Dataset/original/.	3
Figure 2.1	System Overview	15
Figure 2.2	Preprocessing Pipeline	17
Figure 2.3	Training Curves	19
Figure 2.4	Staircase Chart	23
Figure 2.5	Accuracy vs K	25
Figure 2.6	Fold Variance Box Plot	26
Figure 2.7	Heatmap: env (×) strategy vs K	26
Figure 2.8	E1 vs E2 Bar Chart	27
Figure 2.9	Confusion Matrix	29
Figure 2.10	Left: Correct prediction (<i>P. polonicum</i> DTO 148-D1). Right: Incorrect prediction (<i>P. melanoconidium</i> DTO 158-D1 misclassi- fied as <i>P. aurantiogriseum</i>).	30

LIST OF TABLES

Table 1.1	Design Decisions at a Glance	13
Table 2.1	Summary of hand-crafted feature extractors	17
Table 2.2	Deep learning feature extractors: architecture and performance	18
Table 2.3	Fine-tuning hyperparameters	18
Table 2.4	Training convergence summary	18
Table 2.5	Environment evaluation strategies	21
Table 2.6	Dataset statistics	22
Table 2.7	Species and strain distribution	22
Table 2.8	Segmentation approach comparison	23
Table 2.9	Comprehensive accuracy comparison across all feature ex- tractors	24
Table 2.10	Cross-validation top configurations	25
Table 2.11	Per-species accuracy — EfficientNetB1 (Fine-tuned)	28

LIST OF ABBREVIATIONS

Notation	Description
F1	F1 Score
IoU	Intersection over Union
KNN	K-Nearest Neighbors
Qdrant	Vector Database
SDD	Spec-Driven Development
SRS	Software Requirements Specification
TDD	Test-Driven Development

CHAPTER 1. INTRODUCTION

1.1 Problem Statement

1.1.1 Overview

Accurate identification of fungal species is a fundamental task in mycology, with direct implications for food safety, pharmaceutical quality control, and clinical diagnostics. Traditionally, this task relies on expert mycologists who examine colony morphology—color, texture, growth pattern, and pigmentation—across multiple growth media. This manual process is time-consuming, requires years of specialized training, and introduces inter-observer variability. As fungal taxonomy expands and laboratories handle increasing sample volumes, scalable automated approaches become essential.

This thesis addresses the challenge through **MycoAI Retrieval**, an image-based retrieval system for fungal species identification. Given a photograph of a fungal colony grown on a known medium, the system searches a curated reference database and returns the most visually similar known strains along with their species labels. Rather than attempting to classify every possible fungal species in a closed set, the system uses a similarity-based retrieval approach: it extracts visual features from the query image, compares them against pre-computed features of all reference strains, and aggregates the nearest matches to produce a species-level prediction. This design naturally accommodates new species and strains without retraining, a critical advantage in a domain where novel taxa are continuously discovered.

The system is evaluated on a dataset of *Penicillium* species—a genus of significant industrial and medical importance—and demonstrates that retrieval-based classification with learned visual features can achieve high accuracy even with limited training data per species.

1.1.2 Dataset Description

Dataset Structure

The primary dataset consists of 435 high-resolution photographs of fungal colonies cultured on Petri dishes. Each dish contains exactly three colonies of the same strain, grown on a single growth medium and photographed from two angles: **oblique** (top-down angled view, abbreviated `ob`) and **reverse** (view from the bottom of the dish, abbreviated `rev`). The oblique angle captures surface morphology—colony color, texture, and sporulation patterns—while the reverse angle reveals

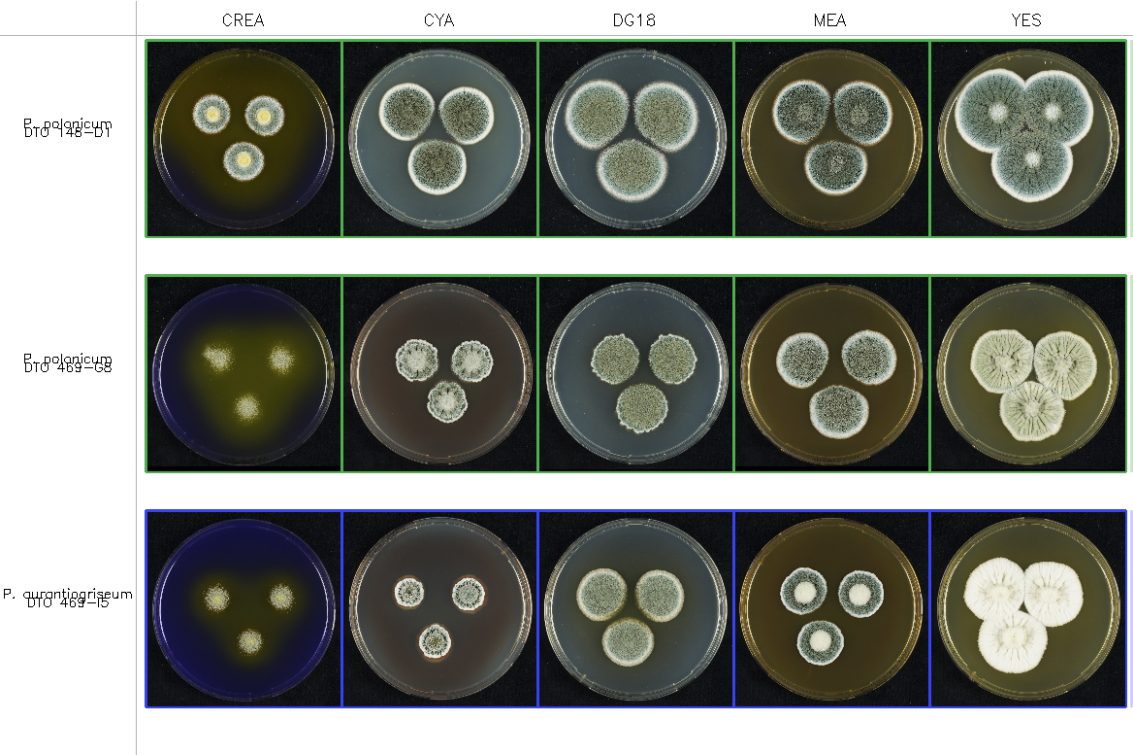
pigments diffused into the agar, which are often species-diagnostic.

The data hierarchy follows the natural biological organization:

```
Species $  
  ightarrow$ Strain $  
    ightarrow$ Growth Medium $  
      ightarrow$ Image (oblique / reverse)
```

- **Species** (8): The taxonomic unit to be predicted. All species belong to the genus *Penicillium*.
- **Strain** (31): A genetically distinct isolate within a species. Each strain is cultured across multiple media.
- **Growth Medium** (7): CREA, CYA, CYA30, CYAS, DG18, MEA, and YES. Different media induce different colony morphologies, and comparing across media is essential for accurate identification.
- **Angle** (2): Oblique (`ob`) and reverse (`rev`), providing complementary visual information.
- **Colonies per dish** (3): Each dish contains three colonies, which are later segmented into individual images.

To illustrate the dataset’s structure, Figure 1.1 shows three strains grown on five representative media from the oblique angle. Rows 1–2 show strains of the same species (*P. polonicum*)—green borders mark same-species colonies, while the third row (*P. aurantiogriseum*) shows a different species marked with red borders.



Hình 1.1: Three fungal strains across five growth media (oblique view). Green border = same species as reference strain; red border = different species. All images from Dataset/original/.

Exploratory Data Analysis

To understand the dataset’s composition and identify challenges for machine learning, we performed an exploratory data analysis with the following findings.

Dataset scale. Figure 1.2 summarizes the overall scale: 435 original dish images are segmented into 1,305 individual colony images (3 colonies per dish). The dataset spans 31 strains across 8 species, with all 435 images successfully parsed and labeled.

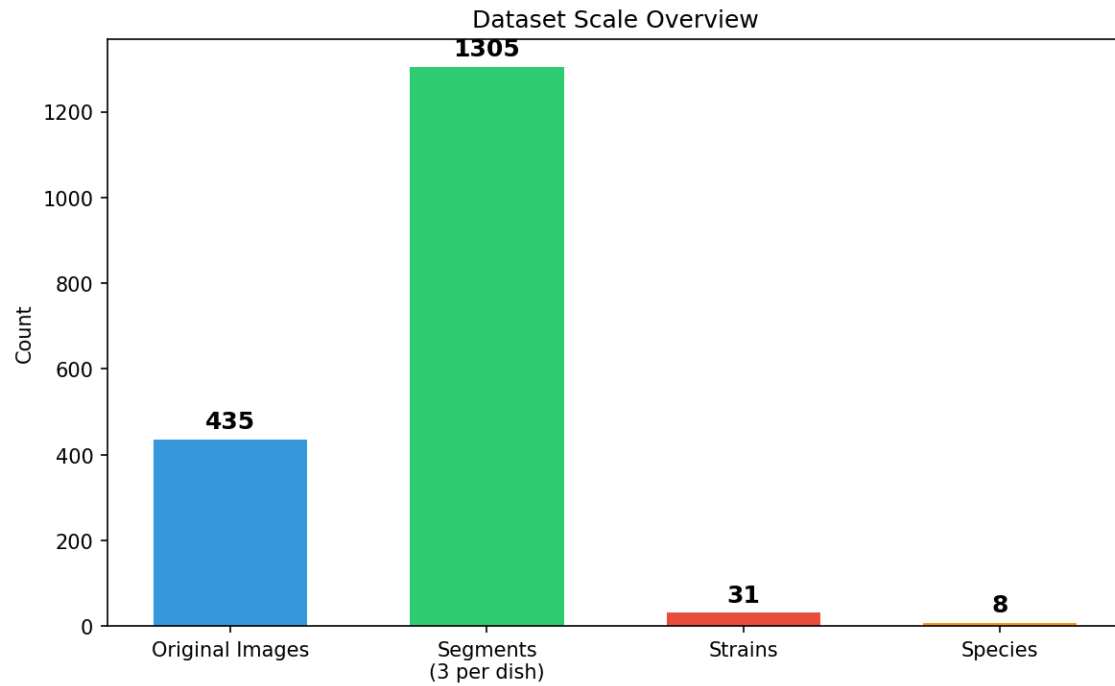


Figure 1.2: Dataset scale overview showing the relationship between original images, segmented colonies, strains, and species.

Species distribution. Figure 1.3 shows the number of original dish images per species. The dataset is **class-imbalanced**: *P. polonicum* contributes 99 images (22.8% of the total), while *P. cyclopium* contributes only 14 images (3.2%). This imbalance reflects real-world collection constraints—some species are more frequently isolated than others—but it creates challenges for supervised learning, as minority classes have insufficient examples for training robust classifiers.

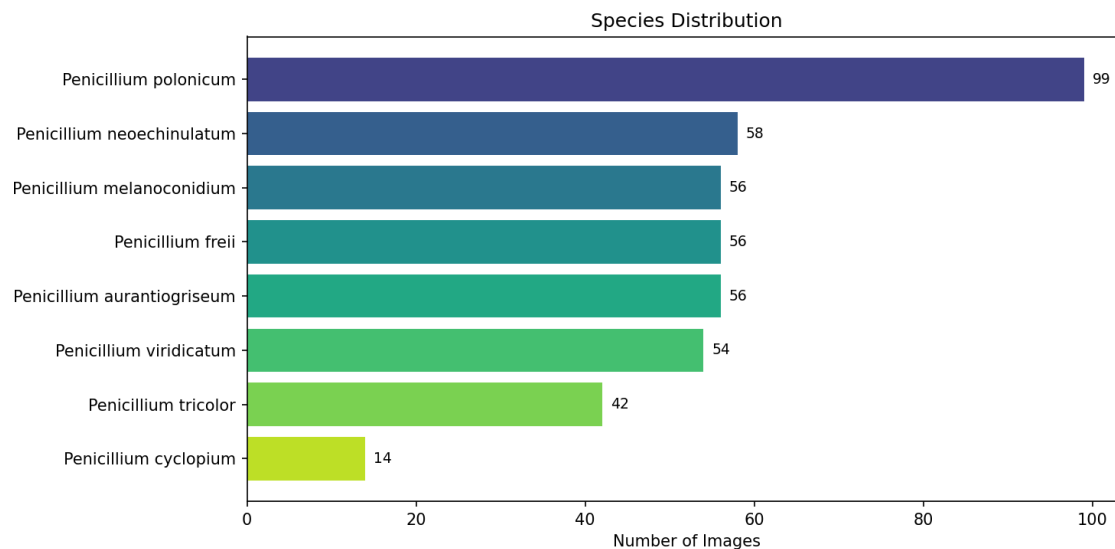


Figure 1.3: Number of original dish images per species in the curated dataset.

Strain-level distribution. Figure 1.4 breaks down the data at the strain level, colored by species. Most strains have exactly 14 images (7 media $\times$ 2 angles), with minor variations due to contaminated samples or missing captures. The per-strain

consistency is a deliberate experimental design choice, ensuring that each strain contributes comparable evidence for both reference indexing and evaluation.

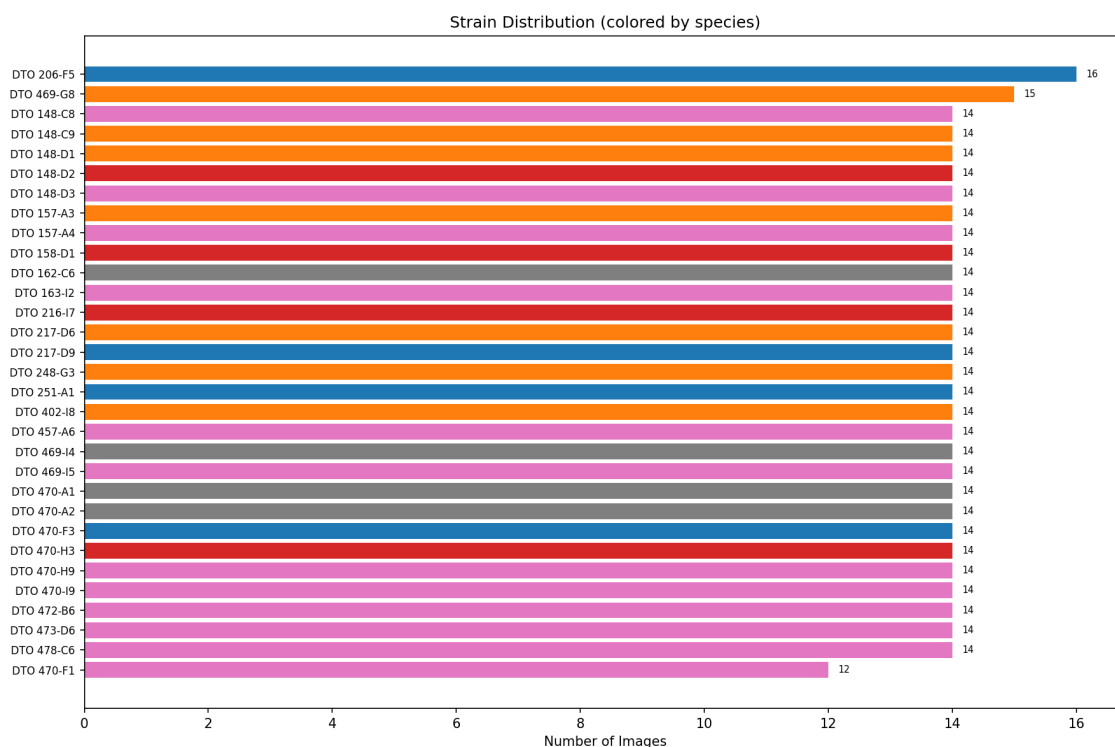


Figure 1.4: Images per strain, colored by species. Most strains contribute 14 images (7 media $\times$ 2 angles).

Growth media coverage. Figure 1.5 shows that images are evenly distributed across the seven growth media, with each medium contributing between 60 and 64 images. This balanced coverage is important because colony appearance can vary dramatically across media—a species that looks distinctive on CYA may appear nearly identical to another species on DG18. The system must learn features that are robust to these media-induced variations.

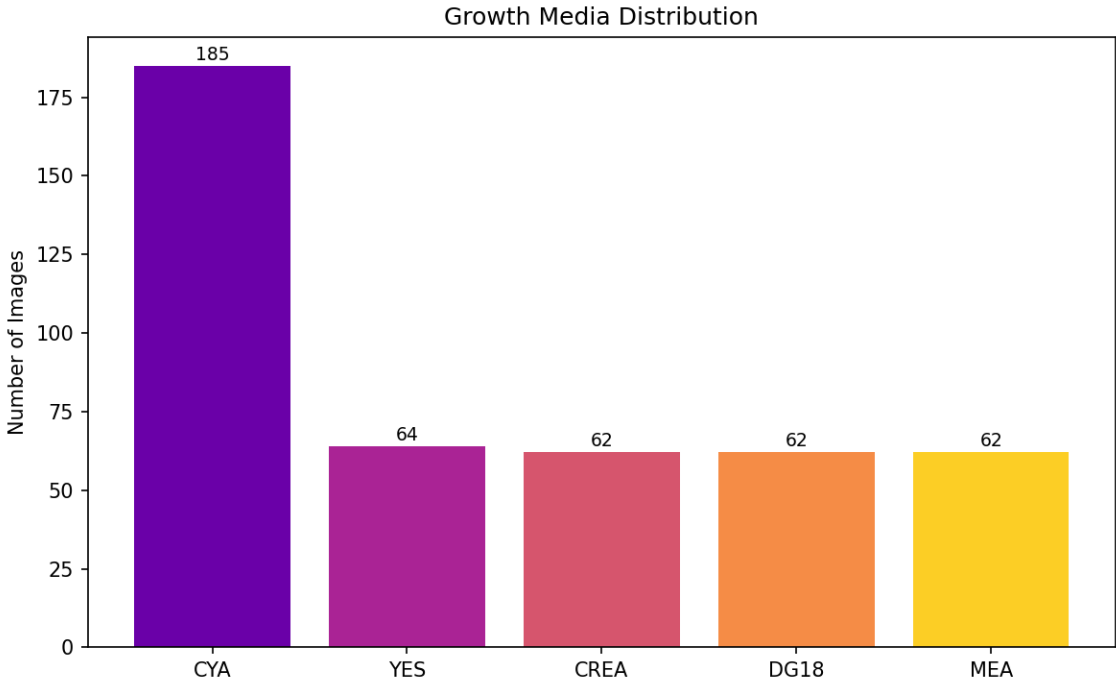


Figure 1.5: Distribution of images across seven growth media.

Media $\times$ species interaction. Figure 1.6 provides a heatmap of image counts per species per medium. The near-uniform cell values confirm that the dataset is well-balanced across media for most species. *P. polonicum* has slightly more images per medium due to having more strains. This balanced design means that species-level evaluation is not confounded by unequal media representation.

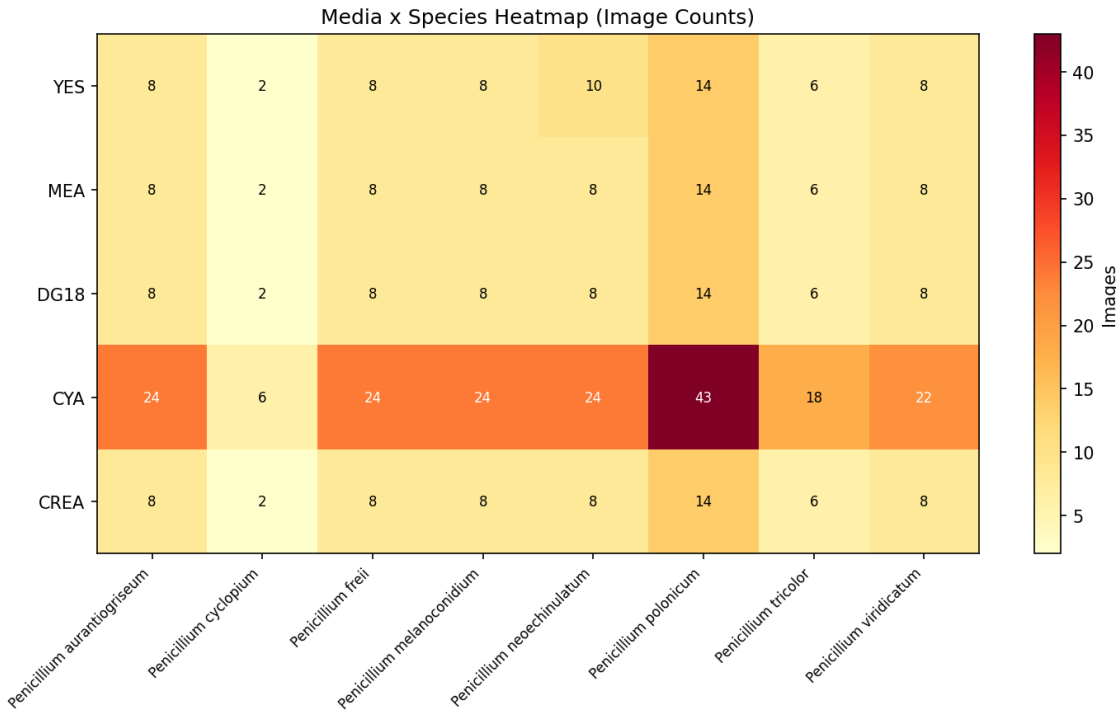


Figure 1.6: Heatmap showing the number of images per species per growth medium.

The Few-Shot Classification Challenge

The dataset’s most critical characteristic is the mismatch between the number of classes and the number of examples per class. Figure 1.7 visualizes this: 8 species distributed across only 31 strains, with a mean of 54 images per species and a median of 56.

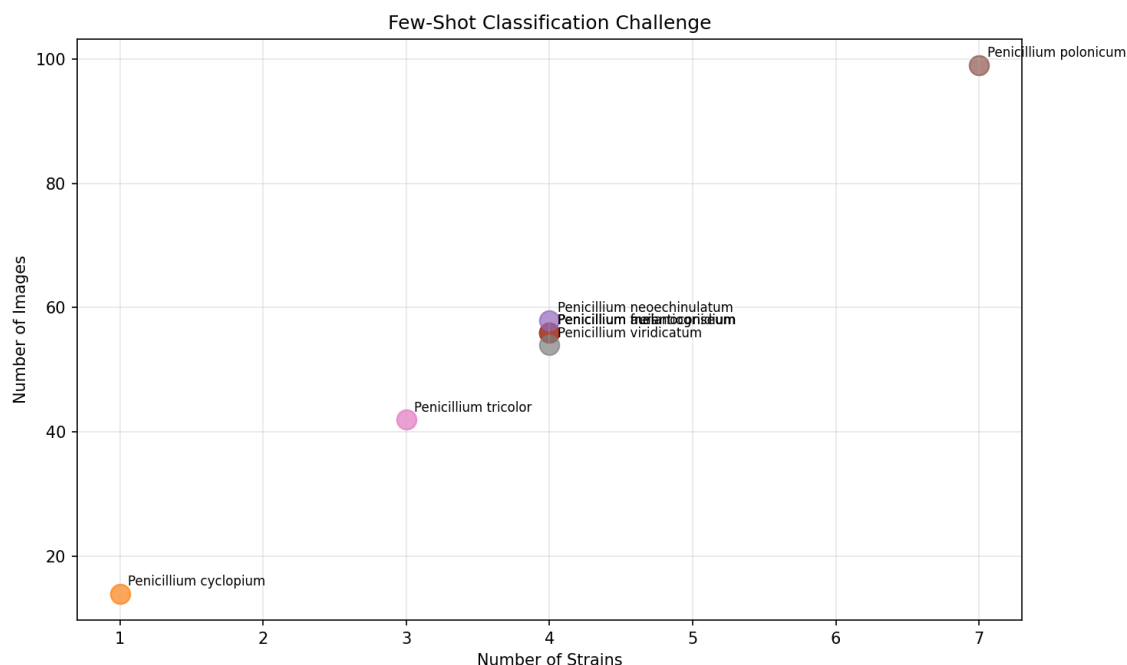


Figure 1.7: Few-shot classification challenge. Each point represents one species, positioned by its number of strains (x-axis) and number of images (y-axis).

This places the problem squarely in the **few-shot classification** regime. Traditional deep learning classifiers—such as a standard CNN with a softmax output layer—typically require hundreds to thousands of examples per class to generalize reliably. With only 14–99 original images per species (and 42–297 segmented colonies), training a conventional classifier from scratch on this dataset would lead to severe overfitting. Transfer learning from models pre-trained on large natural-image datasets (e.g., ImageNet) partially mitigates this, but the domain gap between natural images and fungal colony microscopy remains substantial.

The few-shot challenge is compounded by **strain-level evaluation**: the test set consists of entirely held-out strains (7 strains, one per species except *P. cyclopium*, which lacks a test strain). This means the model must generalize not just to new images of seen strains, but to genetically distinct isolates that may exhibit different morphological characteristics even within the same species. A strain-level split is significantly more demanding than a random image-level split, and it reflects the true deployment scenario where a user submits a sample from a previously unseen strain.

These constraints motivate the choice of a **retrieval-based approach** over a fixed classifier, as elaborated in Section 1.5.

1.1.3 Scope of Work

The scope of this thesis encompasses three interconnected domains:

1. **Algorithmic Research:** Design and optimization of a retrieval pipeline for fungal species identification. This includes colony segmentation from Petri dish images, feature extraction using both hand-crafted descriptors and fine-tuned deep learning models, vector indexing and similarity search, and a weighted aggregation strategy for converting segment-level retrieval results into strain-level species predictions. The research also investigates the effect of growth medium selection and cross-validation strategies on retrieval accuracy.
2. **System Engineering:** Implementation of a production-oriented web application that wraps the retrieval pipeline behind a user interface. This includes authenticated access for researchers, a species catalog management interface, real-time image upload with retrieval feedback, and visualization of prediction results showing the query image alongside its nearest reference neighbors.
3. **Process Innovation:** Application of an agentic engineering methodology to the research workflow. A multi-agent system autonomously proposes experimental hypotheses, executes evaluation runs, and surfaces the best-performing configurations. Agent details are deferred to Chapter 4.

Out of Scope

To maintain focus, the following areas are explicitly excluded from this thesis:

- **De novo species discovery:** The system identifies samples as belonging to known species or rejects them as unknown. It does not propose new taxonomic groupings or discover previously undescribed species.
- **Training of deep learning segmentation models:** The colony segmentation step uses classical computer vision techniques (K-Means clustering and contour detection). Training detection or segmentation models (e.g., YOLO, SAM) for colony extraction is a separate research direction.
- **Mobile or edge deployment:** The system targets server-side deployment. Optimizing models for mobile inference or on-device processing is not addressed.
- **Real-time video classification:** The system processes static images. Video-based colony growth monitoring and temporal analysis are out of scope.
- **Multi-modal fusion with genomic data:** This thesis uses visual features ex-

clusively. Integrating DNA sequence data or metabolomic profiles for hybrid identification is not explored.

- **Growth time metadata:** The current dataset does not include timestamps or growth-stage annotations. Colony morphology evolves over days of incubation, but this temporal dimension is not modeled.
- **Full laboratory information management system (LIMS):** The web application provides species search and prediction. It is not a complete LIMS with sample tracking, workflow automation, or instrument integration.
- **Multi-tenant and organization-level workspaces:** The system is designed for a single laboratory, as described in the Preface—it addresses the specific requirements of the MycoAI lab and its mycologist collaborators. Multi-tenant architectures (supporting multiple independent labs with separate reference databases, user hierarchies, and access control policies) are explicitly out of scope. Extending the system to an organization-wide platform would require significant architectural changes to the data model, authentication system, and vector database partitioning.

1.1.4 Proposed Solution

The proposed solution is a **retrieval-based classification pipeline** that identifies fungal species by comparing visual features against a pre-indexed reference database. Figure 1.8 illustrates the high-level architecture.

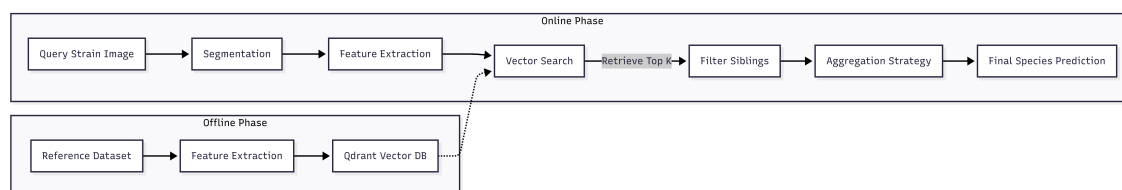


Figure 1.8: High-level architecture of the retrieval-based classification pipeline. A query image passes through segmentation, feature extraction, vector search, and aggregation stages to produce a species prediction.

The pipeline operates in two phases:

Indexing Phase (offline). All reference strain images are processed through the pipeline: each Petri dish image is segmented into individual colony crops, a feature extractor converts each crop into a high-dimensional vector (embedding), and all embeddings are stored in a vector database along with metadata (strain, species, medium, angle). This phase is performed once and updated incrementally when new reference strains are added.

Query Phase (online). When a user submits a new strain for identification:

1. **Segmentation:** The Petri dish image is preprocessed (resized, background-masked via circle detection) and individual colonies are extracted using K-Means clustering. Each dish yields three colony segment images.
2. **Feature Extraction:** Each segment is passed through a feature extractor—a convolutional neural network fine-tuned on the fungal dataset—producing a normalized embedding vector. Multiple complementary extractors can be used; the system supports both hand-crafted descriptors (capturing shape, texture, and color) and deep learning backbones (capturing learned visual features).
3. **Nearest-Neighbor Search:** Each query embedding is compared against the vector database using cosine similarity. The system retrieves the k most similar reference segments, excluding any segments belonging to the query strain itself (sibling filtering) to prevent data leakage.
4. **Aggregation:** Since a strain yields multiple query segments (from different media and angles), the per-segment retrieval results are combined using a weighted voting scheme. Each candidate species receives a score proportional to the sum of similarity scores of its retrieved neighbors, normalized by the frequency of appearance. The species with the highest weighted score is the final prediction.
5. **Result Presentation:** The user sees the predicted species, a confidence score, and the top retrieved reference images for visual inspection and validation.

A full treatment of each pipeline stage—including segmentation algorithms, feature extractor architectures, aggregation formulas, and environment selection strategies—is presented in Chapter 2.

Why a Retrieval Pipeline?

The choice of a retrieval-based design over a conventional end-to-end classifier is deliberate and driven by the characteristics of the problem:

Open-set capability. A classifier with a fixed softmax output can only predict among the species seen during training. Adding a new species requires modifying the architecture and retraining the entire model. In contrast, a retrieval system treats classification as a nearest-neighbor search: adding a new species requires only indexing its reference images into the vector database. No retraining is needed. This is critical in mycology, where the set of known species continuously expands.

Interpretability. The system presents the user with the actual reference images that are most similar to the query. A mycologist can visually compare the query

colony against its nearest neighbors and assess whether the prediction is plausible. This transparency is valuable in a domain where expert judgment remains the gold standard and erroneous identifications can have consequences.

Data efficiency. Retrieval-based classification can perform well with few examples per class because it does not need to learn a decision boundary that separates all classes simultaneously. Instead, it relies on the quality of the feature representation and the assumption that similar images belong to similar species. The feature extractor is trained with a classification objective but deployed as an embedding function, allowing the system to benefit from supervised fine-tuning without being constrained by the number of output classes.

Graceful degradation. When the system encounters a species not in the reference database, the retrieval results will show low similarity scores across all known species. This naturally enables an open-set rejection mechanism: if the highest similarity score falls below a learned threshold, the system can report “unknown species” rather than forcing an incorrect classification.

1.1.5 Rationale

Retrieval over Fixed Classification

A standard supervised classifier maps an input image directly to a probability distribution over N predefined species. This approach has two fundamental limitations for our use case. First, it is **closed-world**: the model cannot express uncertainty about species it was not trained on. Second, it **requires retraining** whenever the species catalog changes, which is impractical in a dynamic research environment.

A retrieval pipeline addresses both issues. The species catalog is decoupled from the model—it lives in the vector database as indexed reference embeddings. Updating the catalog is a database operation, not a machine learning operation. Moreover, retrieval results inherently carry a similarity score, which can be thresholded to implement open-set rejection: if no reference image is sufficiently similar to the query, the system reports “unknown” rather than forcing a potentially incorrect match.

Modularity and Versatility

A key architectural advantage of the retrieval pipeline is its **modularity**. Each stage—segmentation, feature extraction, vector search, and aggregation—is an independent component with a well-defined interface. This has several practical benefits:

Interchangeable feature extractors. The system can use any feature extractor that produces a fixed-length normalized vector, regardless of whether it is a hand-crafted descriptor (HOG, Gabor filters, color histograms) or a deep learning backbone (ResNet, EfficientNet, Vision Transformer). Different extractors capture different visual properties—texture, shape, color, learned features—and can be combined through late fusion or ensemble voting. When a better feature extractor becomes available, it can replace the existing one without modifying any other component.

Interchangeable segmentation models. The segmentation stage is similarly modular. The current system uses K-Means clustering and contour detection, but these can be swapped for deep learning detectors without affecting downstream components. The only contract is that the segmentation stage outputs cropped colony images; how those crops are produced is an implementation detail.

Experiment-friendly design. The modular architecture enables systematic experimentation. A researcher can evaluate the effect of a new feature extractor by running the query phase with different extractor configurations while keeping all other components fixed. This makes the system well-suited for the agentic autoresearch workflow described in Chapter 4, where an autonomous agent iteratively tests hypotheses about pipeline configurations and surfaces the best-performing combinations.

Deployment flexibility. The separation of feature extraction from vector search means the computationally intensive part (feature extraction) can run on specialized hardware (GPU), while the search and aggregation logic can run on a standard server. The vector database can scale independently to handle growing reference collections.

Design Decisions at a Glance

Bảng 1.1: Design Decisions at a Glance

Decision		Choice	Rationale
Classification approach	ap-	Retrieval-based (KNN)	Open-set capable, no retraining for new species
Feature learning		Supervised fine-tuning of pre-trained CNNs	Balances data efficiency with domain adaptation
Feature storage		Vector database with cosine similarity	Efficient high-dimensional search with metadata filtering
Metadata storage		Relational database	ACID compliance for structured data (users, species catalog, audit trail)
Evaluation protocol	proto-	Strain-level hold-out	Reflects real deployment: generalization to unseen strains
Segmentation		Classical CV (K-Means, contour)	Deterministic, fast, and sufficient for colony extraction from controlled lab images
Experiment automation	au-	Multi-agent search loop	Systematic hypothesis testing with reproducible logging

CHAPTER 2. FUNGAL SPECIES RETRIEVAL MODEL

2.1 Fungal Species Retrieval Model

2.1.1 Motivation

Traditional fungal identification relies on morphological features observed under a microscope—colony color, texture, growth rate, and sporulation patterns. These features can be subtle and are often subjective when assessed by different mycologists. By utilizing deep learning embeddings, the system can capture complex visual patterns that are invariant to minor lighting or scale changes, providing a “similarity search” that mimics how mycologists compare unknown samples to known reference slides.

The retrieval-based approach offers a key advantage over end-to-end classification: when a new fungal strain is added to the reference database, no model retraining is required. The new strain’s colony images are simply embedded and inserted into the vector index, making the system inherently scalable and suitable for the “long tail” of rare species that plague traditional classifiers.

2.1.2 Related Work

Current state-of-the-art in fungal identification often uses Convolutional Neural Networks (CNNs) such as ResNet~[1], EfficientNet~[2], or Vision Transformers (ViTs). However, these models often struggle with the “long tail” of rare species where few training examples exist. Image retrieval via instance-based learning is more robust for these cases, as it only requires a few reference examples per species.

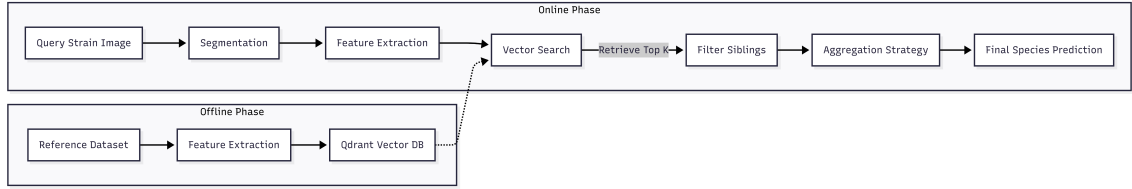
Recent work on content-based image retrieval (CBIR) for biological specimens demonstrates that combining hand-crafted features (HOG, Gabor filters, color histograms) with deep features can yield complementary improvements. Vector databases such as Qdrant enable low-latency cosine similarity search over high-dimensional embeddings, making retrieval-based classification practical at scale.

2.1.3 Methodology

The retrieval pipeline consists of four main stages: (1) preprocessing and colony segmentation, (2) feature extraction and embedding, (3) vector database indexing and retrieval, and (4) multi-image aggregation for strain-level prediction.

Preprocessing Pipeline

Raw Petri dish images undergo a multi-stage preprocessing pipeline to isolate individual fungal colonies. Given an original microscopy image $I_{\text{raw}} \in \mathbb{R}^{H \times W \times 3}$, the pipeline operates as follows:



Hình 2.1: System Overview

Stage 1 — Full-Image Preprocessing: 1. **Resizing:** Images are standardized to (256×256) pixels to ensure consistent input dimensions. 2. **Circle Detection:** The Hough Circle Transform detects the circular Petri dish boundary, producing a center (c_x, c_y) and radius (R) . 3. **Background Masking:** Pixels outside the detected dish are masked to black, removing irrelevant background (labels, shadows, bench surface). 4. **Cropping:** The dish region is extracted, discarding the remainder.

Stage 2 — Colony Segmentation:

Three complementary approaches were developed for colony segmentation:

K-Means Clustering Segmentation The image is converted from RGB to HSV color space to better separate fungal colonies from the agar medium. A Gaussian blur with kernel size (9×9) and $(\sigma = 1.5)$ reduces high-frequency noise. K-Means clustering partitions pixels into foreground (colony) and background (agar) clusters.

A key innovation is the **Local K=2 Shrink** method, which addresses a specific failure mode: bright small colonies that produce light halos on certain media (notably MEA and YES). The flare phenomenon occurs when the agar near a colony appears bright, causing K-Means to group the halo with the colony foreground rather than the background. The Local K=2 method operates as follows:

For each candidate colony region (R_i) in HSV space: 1. Apply a second K=2 clustering on the V (value) channel pixels within (R_i) only. 2. Identify the bright cluster (higher mean V value) and the dim cluster. 3. Construct a mask retaining only pixels belonging to the bright cluster. 4. Shrink the mask via morphological erosion to remove borderline halo pixels.

This two-stage approach strips light halos from bright small colonies without affecting larger textured colonies, where the halo is already negligible relative to colony area.

Limitation: On plates with strong agar flare (e.g., certain YES medium images where the agar itself reflects light unevenly), K-Means occasionally misclassifies

the flare region as foreground. This motivated exploration of the contour-based and YOLO-based alternatives described below.

Contour-Based Segmentation An alternative segmentation pipeline based on edge detection was developed to bypass K-Means sensitivity to lighting:

1. **Canny Edge Detection:** Applied to the blurred grayscale image with thresholds ($t_{\text{low}} = 30$), ($t_{\text{high}} = 80$).
2. **Morphological Closing:** Dilation ((5×5) kernel, 3 iterations) followed by erosion ((3×3) kernel, 2 iterations) to seal gaps in colony edges.
3. **Circularity Filter:** Contours are scored by ($\text{area} \times \text{circularity}$), where circularity is defined as $4\pi A/P^2$ (A = contour area, P = perimeter). A perfect circle scores 1.0. Contours are ranked and the top-3 are selected per dish, subject to area constraints ($400 \leq A \leq 23,700 \text{ px}^2$).
4. **Bounding Box Extraction:** For each selected contour, the axis-aligned bounding rectangle is computed; boxes with area below the minimum threshold are filtered.

This approach is more robust to uneven illumination but less reliable when colonies touch or overlap.

YOLO-Based Segmentation To leverage modern deep learning for the segmentation task, a YOLOv8 instance segmentation model was trained on a relabeled dataset:

- **Dataset:** `Dataset/manual_labeled_data_roboflow_species/`, annotated with colony bounding boxes and species labels derived from `Dataset/strain_to_`
- **Split:** Deterministic random train/test split with no separate validation partition.
- **Model:** YOLOv8n-seg (nano variant, ($\sim 3.2\text{M}$ parameters)) trained for 50 epochs with image size 640, batch size 16.
- **Output:** Instance masks and bounding boxes for each detected colony, along with a confidence score ($p \in [0, 1]$).

YOLO segmentation inherits the train/test split of the underlying labeled dataset, enabling downstream cross-validation experiments where segmentation quality can be assessed per fold.

Feature Extraction and Embedding

Each segmented colony image is passed through a feature extractor ($f_\theta : \mathbb{R}^{256 \times 256 \times 3} \rightarrow \mathbb{R}^d$) that maps the image to a high-dimensional embedding vector. All extracted



Hình 2.2: Preprocessing Pipeline

features are L2-normalized to unit length for cosine similarity comparison:

$$\mathbf{v} = \frac{f_{\theta}(\mathbf{x})}{\|f_{\theta}(\mathbf{x})\|_2}$$

Hand-Crafted Extractors:

Bảng 2.1: Summary of hand-crafted feature extractors

Extractor	Dimension d	Captures	Accuracy
HOG	3,780	Edge orientation, shape structure	52%
Gabor	40	Texture at multiple frequencies/orientations	55%
ColorHist (RGB)	96	RGB color distribution (32 bins/channel)	–
ColorHistHS	64	Hue/Saturation profile (32 bins each)	65%

Deep Learning Extractors:

Three CNN architectures were evaluated, both pretrained on ImageNet-1K and fine-tuned on the fungal dataset:

The hybrid HS+ResNet50 extractor concatenates ColorHistHS (64-dim) and ResNet50 (2,048-dim) features, with HS weighted $3.0(\times)$ before concatenation to

Bảng 2.2: Deep learning feature extractors: architecture and performance

Model	Parameters	Dim d	Accuracy	vs. Pretrained
ResNet50 (Pretrained)	25.6M	2,048	63%	baseline
ResNet50 (Fine-tuned)	25.6M	2,048	78.6%	+15.6%
MobileNetV2 (Pretrained)	3.5M	1,280	60%	baseline
MobileNetV2 (Fine-tuned)	3.5M	1,280	78.6%	+18.6%
EfficientNetB1 (Pretrained)	7.8M	1,280	63%	baseline
EfficientNetB1 (Fine-tuned)	7.8M	1,280	83.3%	+20.3%
HS+ResNet50 (Hybrid)	–	2,112	72%	–

balance the contribution of color and texture modalities.

Fine-Tuning Methodology:

The three CNN backbones were fine-tuned on the fungal dataset using supervised classification as a proxy task. Key training parameters:

Bảng 2.3: Fine-tuning hyperparameters

Parameter	Value	Rationale
Batch size	16	Memory/gradient stability balance
Learning rate	0.0001	Conservative; prevents catastrophic forgetting
Optimizer	Adam	Adaptive learning, robust convergence
Loss function	CrossEntropyLoss	Standard multi-class
Max epochs	50	Early stopping prevents overtraining
Early stopping patience	10	Halts when validation plateaus
Augmentation	HFlip(0.5), Rot $\pm 10^\circ$	Moderate to preserve morphology

After training, the classification head is discarded; only the backbone encoder is retained for feature extraction. The training set comprised 1,011 images from 24 strains (8 species); the validation set used 294 images from 7 held-out test strains.

Bảng 2.4: Training convergence summary

Model	Training Time	Val Accuracy	Convergence Epoch
ResNet50	~2 h	78.6%	35
MobileNetV2	~1.5 h	78.6%	25
EfficientNetB1	~2 h	83.3%	35

Alternative: Triplet Loss Training:

A contrastive training approach using triplet margin loss was explored as a theoretically better fit for retrieval tasks. For each anchor image x_a , a positive x_p (same



Hình 2.3: Training Curves

species) and negative $\mathbf{x}_n$ (different species) form a triplet. The loss optimizes:

$$\mathcal{L}_{\text{triplet}} = \max(0, \|f(\mathbf{x}_a) - f(\mathbf{x}_p)\|_2 - \|f(\mathbf{x}_a) - f(\mathbf{x}_n)\|_2 + \alpha)$$

with margin $\alpha = 1.0$ and embedding dimension $d = 128$.

Result: Triplet loss achieved only 64.3% accuracy, a 19% drop from cross-entropy fine-tuning. Analysis identified several causes:

- Small dataset (1,011 images): triplet loss typically requires $10\times$ more data than classification.
- Reduced embedding dimension (128 vs. 1,280) limits representational capacity.
- Random negative sampling without hard negative mining fails to push apart visually similar but different species.
- Class imbalance: *P. polonicum* (6 strains) dominates triplet sampling, biasing predictions.

The cross-entropy fine-tuning approach was therefore selected as the production method.

Vector Database Retrieval

All 1,305 segmented colony images from training strains are embedded and indexed in a Qdrant vector database. The database supports cosine similarity search with dynamic metadata filtering:

- **Collections:** `myco_fungi_features_full` (pretrained) and `myco_fungi_features_f` (fine-tuned).
- **Payload:** Strain ID, species label, growth medium, camera angle (ob/reverse), segment index.
- **Multi-vector support:** Each image point can store vectors from multiple extractors simultaneously (ResNet50, EfficientNetB1, etc.).
- **Dynamic filtering:** During evaluation, vectors belonging to the query strain are excluded to prevent data leakage ("sibling filtering").

For a query image $\mathbf{x}_q$ with feature $\mathbf{v}_q$, retrieval returns the top- k nearest neighbors:

$$\mathcal{N}_k(\mathbf{v}_q) = \operatorname{argmax}_{\mathbf{v} \in \mathcal{D} \setminus \mathcal{D}_{\text{strain}}}^{(k)} \frac{\mathbf{v}_q \cdot \mathbf{v}}{\|\mathbf{v}_q\| \cdot \|\mathbf{v}\|}$$

where $\mathcal{D}$ is the full database and $\mathcal{D}_{\text{strain}}$ is the subset of vectors from the test strain.

Aggregation Strategy

A single strain produces multiple query segments (typically 18 segments: 3 colonies $\times$ 2 angles $\times$ 3 media conditions). Each segment independently retrieves (k) neighbors. Aggregation combines these results for a single strain-level prediction.

Let the strain produce (m) query segments, and let the (i)-th segment retrieve (k) neighbors with species labels ($c_{\{i,1\}}, \dots, c_{\{i,k\}}$) and similarity scores ($s_{\{i,1\}}, \dots, s_{\{i,k\}}$).

Weighted Aggregation (default): For each candidate species (c),

$$S_c^{\text{weighted}} = \frac{\sum_{i=1}^m \sum_{j=1}^k s_{i,j} \cdot \mathbf{1}[c_{i,j} = c]}{\sum_{i=1}^m \sum_{j=1}^k \mathbf{1}[c_{i,j} = c]}$$

This normalizes for species that may appear frequently but with low similarity (e.g., common but visually dissimilar species).

Uniform Aggregation: Simple voting counts occurrences:

$$S_c^{\text{uni}} = \frac{1}{m \cdot k} \sum_{i=1}^m \sum_{j=1}^k \mathbf{1}[c_{i,j} = c]$$

The weighted strategy is more robust to outliers and is used as the default.

The final predicted species is:

$$\hat{c} = \arg \max_c S_c$$

with confidence $\text{conf}(\hat{c}) = S_{\hat{c}}$.

Environment Selection Strategies

To evaluate robustness across growth conditions, four testing strategies are employed:

Bảng 2.5: Environment evaluation strategies

Strategy	Training Set	Test Set	Purpose
E1 (All)	All media	All media	Standard benchmark
E2 (Balanced)	All media	Equal per-media samples	Fair per-condition evaluation
E3 (Single-Env)	All media	One specific medium	Test single-condition robustness
E4 (Leave-One-Out)	All media	All except one medium	Test generalization with missing media

Key finding: E1 performs best. t-SNE visualizations (Section 2.4.6) confirm that fine-tuned models learn environment-invariant features, so training with all environments provides maximum diversity without introducing confusion.

Ensemble Methods

Multiple feature extractors can be combined via weighted score fusion. Given extractors $(f_1, f_2, \dots, f_n)$ with per-extractor scores $(S_c^{(t)})$ and weights (w_t) :

$$S_c^{\text{ensemble}} = \sum_{t=1}^n w_t \cdot S_c^{(t)}$$

Analysis of complementary cases (where one extractor succeeds but another fails) informs weight tuning. For instance, EfficientNetB1 excels at texture discrimination while ColorHistHS captures color profiles critical for separating *P. tricolor* (greenish) from *P. viridicatum*.

2.1.4 Experiments and Results

Dataset and Split Strategy

Bảng 2.6: Dataset statistics

Metric	Value
Total Petri dish images	435
Processed successfully	435 (100%)
Total colony segments	1,305
Failed segmentations	0
Species	8 <i>Penicillium</i> species
Training strains	24 (1,011 segments)
Test strains	7 (294 segments)

The split is performed at the **strain level**: entire strains are held out from training, ensuring the model generalizes to novel strains rather than memorizing known ones. Test strains are strictly excluded from all training, fine-tuning, and database indexing. During retrieval evaluation, sibling filtering additionally removes segments from the same parent image as the query.

Bảng 2.7: Species and strain distribution

Species	Train Strains	Test Strain
<i>P. aurantiogriseum</i>	3 (DTO 457-A6, 470-H9, 473-D6)	DTO 469-I5
<i>P. cyclopium</i>	1 (DTO 148-C8)	–
<i>P. freii</i>	3 (DTO 162-C6, 470-A1, 470-A2)	DTO 469-I4
<i>P. melanoconidium</i>	3 (DTO 148-D2, 216-I7, 470-H3)	DTO 158-D1
<i>P. neoechinulatum</i>	3 (DTO 206-F5, 251-A1, 470-F3)	DTO 217-D9
<i>P. polonicum</i>	6 (DTO 148-C9, 157-A3, ...)	DTO 148-D1
<i>P. tricolor</i>	2 (DTO 157-A4, 472-B6)	DTO 470-I9
<i>P. viridicatum</i>	3 (DTO 148-D3, 470-F1, 478-C6)	DTO 163-I2

Note: *P. cyclopium* has only 1 strain and therefore cannot serve as a test strain under the strain-level split; it is used for training only.

Segmentation Experiments

K-Means segmentation parameters were swept to maximize colony extraction quality. The contour-based pipeline was tested as a complementary approach on images with strong agar flare.

The YOLOv8n-seg model was trained on a manually labeled subset. Key metrics:

For downstream retrieval, the K-Means pipeline with Local K=2 Shrink was se-

Bảng 2.8: Segmentation approach comparison

Method	Colonies/Plate	Robust to Flare?	Requires Labels?
K-Means (K=2, HSV)	3	Partial (Local K=2 helps)	No
Contour (Canny + Circ.)	2-3	Yes (edge-based)	No
YOLOv8n-seg	Variable	Yes (learned)	Yes



Hình 2.4: Staircase Chart

lected as the primary segmentation method. The YOLO-based approach is reserved for the cross-validation experiment pipeline where segmentation quality can be assessed per fold.

Retrieval Experiments: Staircase Chart

The primary metric for retrieval experiments is the F1 score, balancing precision and recall across species. An iterative autoresearch loop tested hundreds of combinations of distance metrics, aggregation strategies, and neighbor counts ((k)). Each combination is a single dot on the staircase chart.

The staircase chart (Figure~2.4) plots experiment index on the x-axis and F1 score on the y-axis. Gray dots represent experiments below the running best; green dots set a new best and step the staircase upward. Each green dot is labeled with the formula-algorithm pair that achieved the improvement.

Overall Performance Comparison

Bảng 2.9: Comprehensive accuracy comparison across all feature extractors

Feature Extractor	Type	Accuracy	Improvement
EfficientNetB1 (Fine-tuned)	Deep Learning	83.3%	+20.3%
ResNet50 (Fine-tuned)	Deep Learning	78.6%	+15.6%
MobileNetV2 (Fine-tuned)	Deep Learning	78.6%	+18.6%
HS+ResNet50	Hybrid	72%	—
ColorHistHS	Hand-crafted	65%	—
ResNet50 (Pretrained)	Deep Learning	63%	—
EfficientNetB1 (Pretrained)	Deep Learning	63%	—
MobileNetV2 (Pretrained)	Deep Learning	60%	—
Gabor	Hand-crafted	55%	—
HOG	Hand-crafted	52%	—
Triplet Loss (EfficientNetB1)	Deep Learning	64.3%	—

Key insights:

1. **Fine-tuning is critical:** +15–20% improvement over pretrained ImageNet models.
2. **EfficientNetB1 dominates:** 83.3% accuracy with only 7.8M parameters.
3. **MobileNetV2 is efficient:** Matches ResNet50 (78.6%) with $7\times$ fewer parameters (3.5M vs. 25.6M).
4. **Hand-crafted features plateau:** Maximum 65% accuracy, insufficient alone.
5. **Triplet loss underperforms:** 19% worse than cross-entropy, confirming that supervised fine-tuning is superior for small datasets.

Cross-Validation Results

A 5-fold strain-level cross-validation experiment was conducted to assess stability and sensitivity to hyperparameters. Fixed extractor: EfficientNetB1 (Fine-tuned). Factors:

Folds = 5, Env strategies = 2 (E1, E2), Agg strategies = 2 (weighted, uni), $K = \{3, 5, 7, 9, 11\}$

Total: $5 \times 2 \times 2 \times 5 = 100$ runs.

Findings:

- **E1 (all environments)** consistently outperforms E2 (balanced), confirming that environment diversity in training improves generalization.



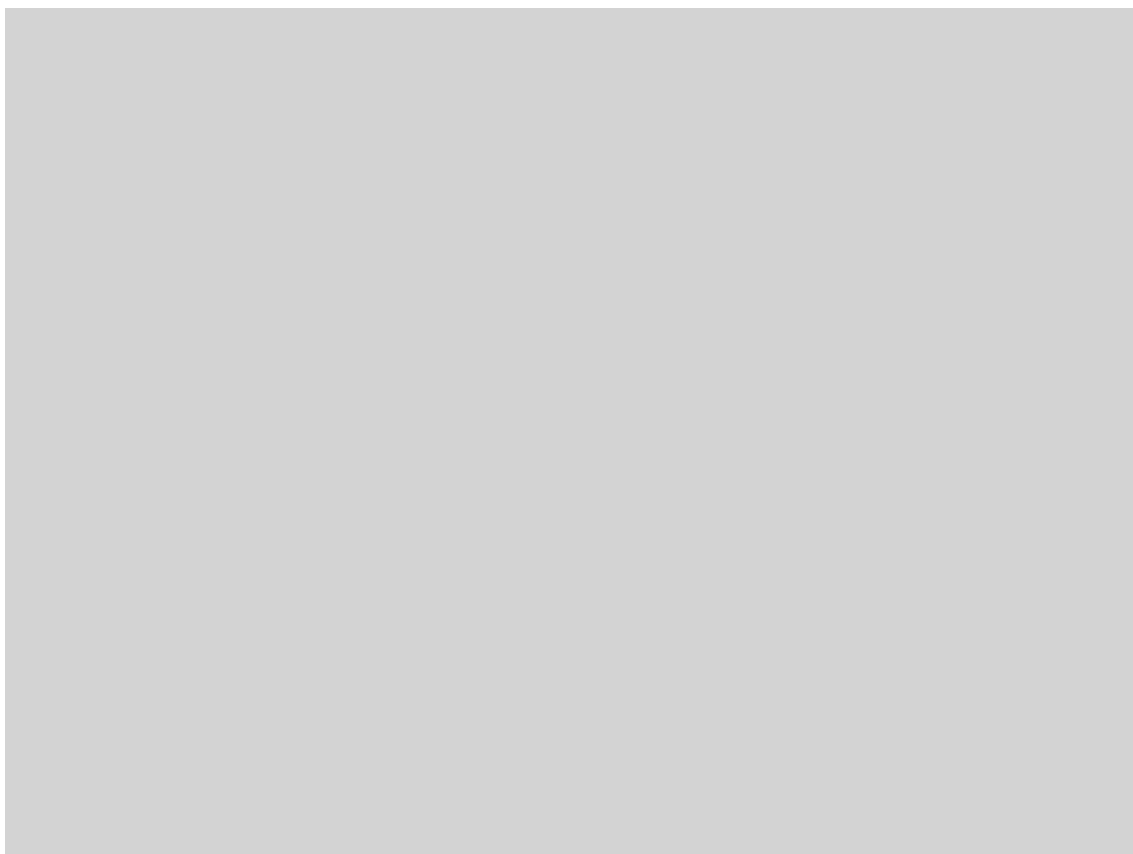
Hình 2.5: Accuracy vs K

Bảng 2.10: Cross-validation top configurations

Env	Agg	K	Mean Acc.	Std	Min	Max
E1	weighted	7	0.833	0.052	0.762	0.905
E1	weighted	9	0.829	0.048	0.762	0.905
E1	uni	5	0.810	0.065	0.714	0.905
E1	uni	7	0.810	0.055	0.714	0.905
E2	weighted	9	0.805	0.072	0.714	0.905

- **Weighted aggregation** outperforms uniform, especially at higher k .
- $k = 7$ is the sweet spot: smaller k is sensitive to noise, larger k risks including dissimilar neighbors.
- **Stability:** Standard deviation across folds is 5–7%, indicating robustness to which specific strains are held out.

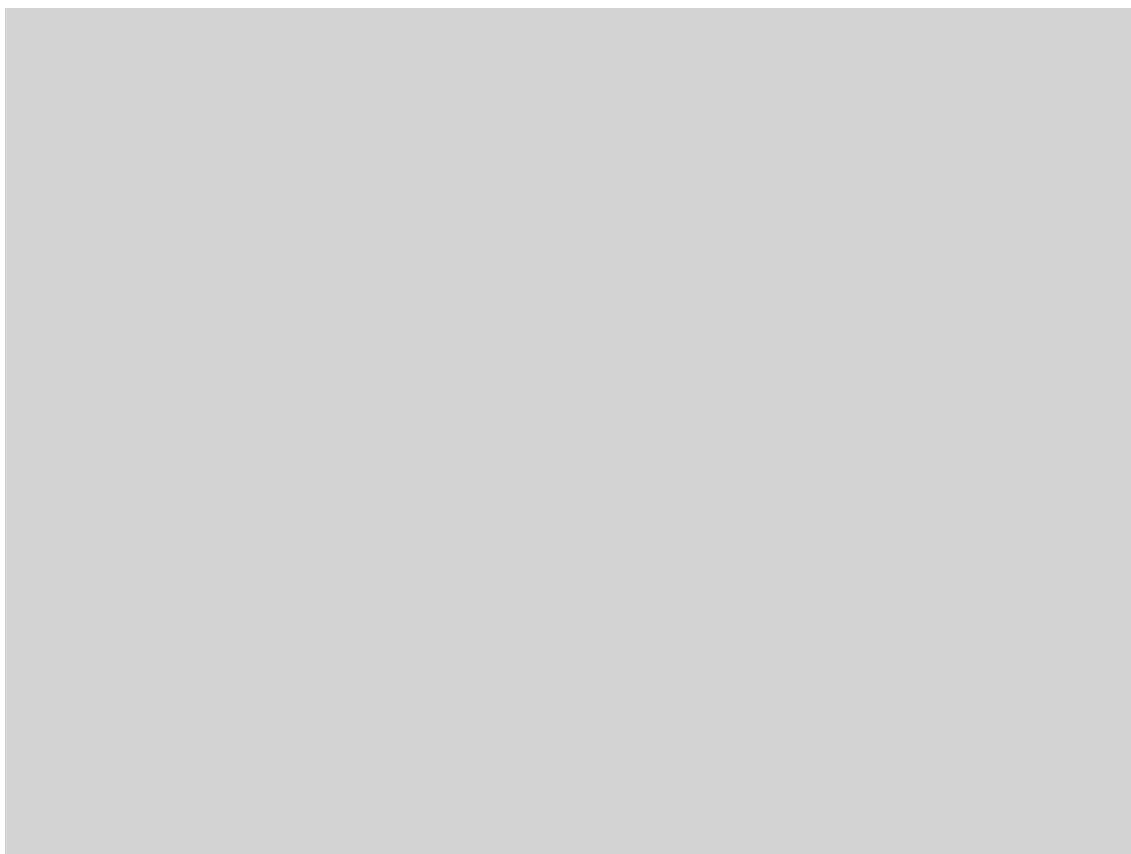
The fold variance box plot shows accuracy distributions for each configuration. E1/weighted configurations exhibit the tightest inter-quartile ranges, confirming stability.



Hình 2.6: Fold Variance Box Plot



Hình 2.7: Heatmap: env (×) strategy vs K



Hình 2.8: E1 vs E2 Bar Chart

Feature Space Analysis (t-SNE)

To understand feature quality, the 2,048-dimensional feature space from fine-tuned ResNet50 was projected to 2D using t-SNE ((t)-parameter = 30, perplexity = 40). Each point represents one segmented colony image.

By Species — The projection reveals clear species-level clustering:

- *P. polonicum*, *P. freii*, and *P. neoechinulatum* form well-separated, compact clusters.
- *P. melanoconidium* and *P. aurantiogriseum* show partial overlap, explaining their high confusion rate.
- *P. tricolor* and *P. viridicatum* are adjacent but distinguishable, consistent with the 67% and 83% per-species accuracies.

By Environment — The same projection colored by growth medium shows **no environment-based clustering**: all media colors are uniformly distributed throughout the space. This is a **positive finding**—it confirms the model successfully learns environment-invariant, species-discriminative features. Training with all environments (E1) therefore maximizes diversity without introducing medium-specific confusion.

Confusion Matrix and Per-Species Analysis

Bảng 2.11: Per-species accuracy — EfficientNetB1 (Fine-tuned)

Species	Test Sets	Correct	Accuracy
<i>P. polonicum</i>	6	6	100%
<i>P. freii</i>	6	6	100%
<i>P. neoechinulatum</i>	6	6	100%
<i>P. aurantiogriseum</i>	6	5	83%
<i>P. viridicatum</i>	6	5	83%
<i>P. tricolor</i>	6	4	67%
<i>P. melanoconidium</i>	6	3	50%
Total	42	35	83.3%

Recurring confusion patterns:

- *P. melanoconidium* ↔ *P. aurantiogriseum*: Both produce dark colonies, causing reciprocal misclassification. *P. melanoconidium* is the hardest species (50%).
- *P. tricolor* ↔ *P. viridicatum*: Shared greenish pigmentation leads to bidirectional confusion.
- *P. aurantiogriseum* ↔ *P. polonicum*: Under certain growth conditions, similar texture and edge patterns cause rare misclassifications.

Prediction Examples

Visualizations show the query colony image (left) alongside its top-7 retrieved neighbors with similarity scores. Successful predictions (accuracy $\geq 83\%$) show high confidence (≥ 0.30) and visually consistent neighbors. Failed predictions show lower confidence (0.15–0.25) and mixed species among retrieved neighbors.

2.1.5 Discussion

The results demonstrate that a retrieval-based classification system with fine-tuned deep features can achieve 83.3% strain-level accuracy across 8 *Penicillium* species. Several findings merit discussion:

Fine-tuning is the single most impactful factor. The 15–20% improvement over ImageNet-pretrained features shows that domain adaptation—even with a modest 1,011 training images—is essential for microscopy images. The domain gap between natural photographs (ImageNet) and fungal colony microscopy is substantial.

Environment-invariant learning emerges naturally. The t-SNE analysis confirms that fine-tuned models ignore growth medium variations, a critical require-



Hình 2.9: Confusion Matrix

ment for real-world deployment where a query strain may be cultivated on any medium. This justifies the E1 (all environments) training strategy.

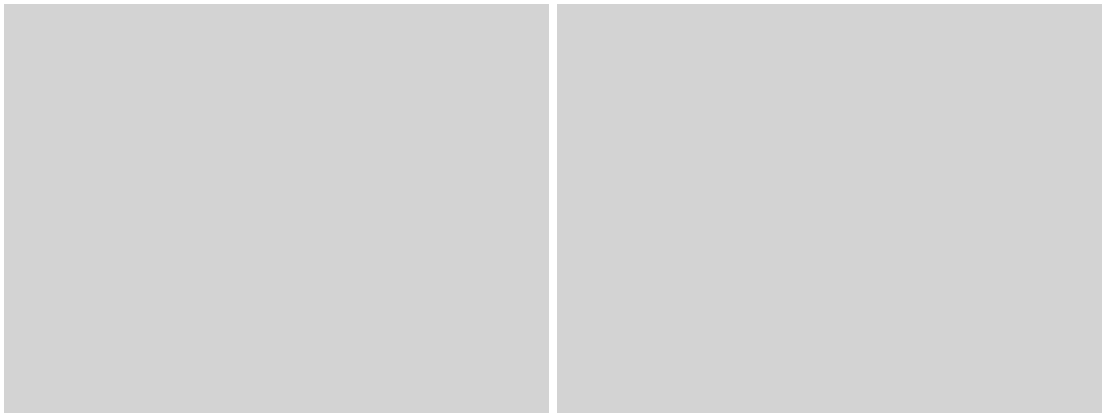
Efficient architectures are competitive. MobileNetV2 matches ResNet50 with 7(×) fewer parameters, making it suitable for edge deployment. EfficientNetB1 provides the best accuracy at a reasonable computational cost.

The triplet loss result is a valuable negative finding. Despite theoretical alignment with retrieval tasks, triplet loss severely underperformed (64.3% vs. 83.3%). This aligns with literature: triplet loss requires very large datasets and hard negative mining, neither of which was feasible with 1,011 training samples.

Challenging species remain. *P. melanoconidium* at 50% accuracy is the primary bottleneck. Potential solutions include collecting additional strains, implementing hard negative mining, or using a two-stage classifier (genus → species).

Segmentation quality matters. The K-Means Local K=2 Shrink innovation successfully mitigates agar flare on most images, but plates with extreme lighting remain problematic. Future work could integrate the YOLO-based segmentation with larger manually labeled datasets.

Model selection recommendations:



Hình 2.10: Left: Correct prediction (*P. polonicum* DTO 148-D1). Right: Incorrect prediction (*P. melanoconidium* DTO 158-D1 misclassified as *P. aurantiogriseum*).

Use Case	Recommended Model	Accuracy
Maximum accuracy	EfficientNetB1 (Fine-tuned)	83.3%
Balanced performance	ResNet50 (Fine-tuned)	78.6%
Edge/mobile deployment	MobileNetV2 (Fine-tuned)	78.6%

CHAPTER 3. WEB APPLICATION DESIGN & IMPLEMENTATION

3.1 Web Application

3.1.1 Requirements Analysis

The web application translates the MycoAI Retrieval model into a governed, multi-role product supporting two actor classes: **User** (retrieval operator) and **Data Owner** (governance operator). Requirements are defined in `docs/SRS.md` and elaborated across 13 technical design documents in `docs/technical_spec/` and 8 feature specifications in `docs/feature_spec/`.

Actor Model and Permissions

Actor	Role Value	Capabilities
User	<code>user</code>	Upload images, retrieve species predictions, review/edit bounding boxes, download batch results, submit feedback/contribution proposals
Data Owner	<code>owner</code>	All User capabilities plus: index reference data, CRUD species/media metadata, browse/manage dataset, review feedback, re-index Qdrant, assess/promote Candidate Models, manage users and roles

The Data Owner role inherits all User permissions. A permission matrix enforced at both the API layer (403 Forbidden) and the UI layer (conditional rendering) gates every endpoint. Backend RBAC is implemented at `backend/src/mycoai_retri` using `FastAPI Depends (get_current_user)` and `require_role("owner")` dependency chains. The frontend at `frontend/src/App.tsx` conditionally renders owner-restricted pages.

Use Case Diagram

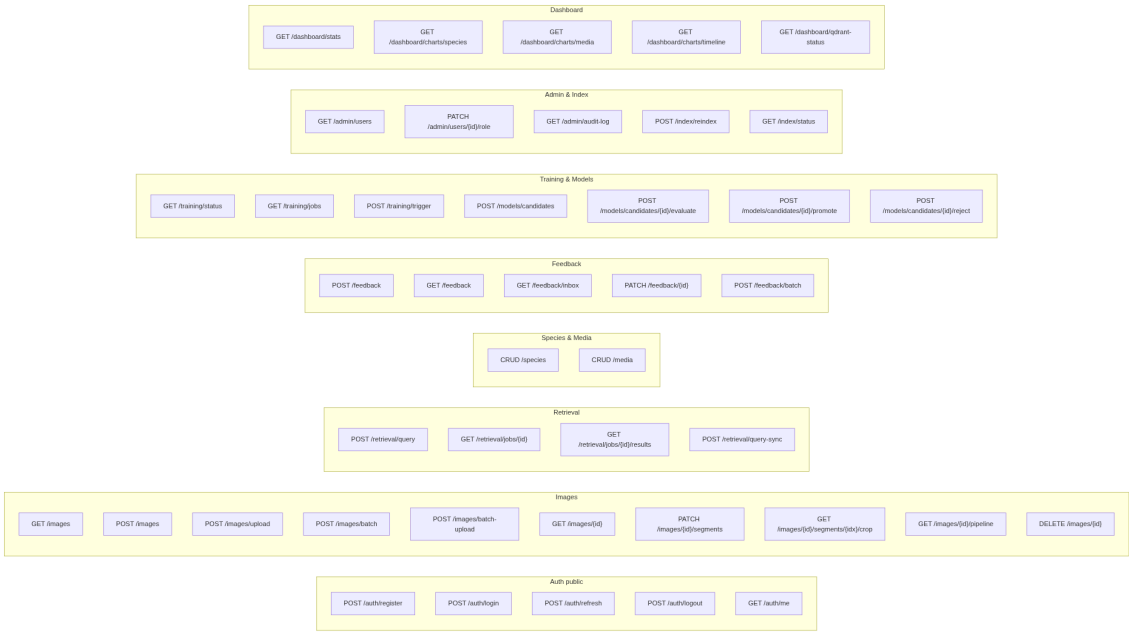


Figure 3.1: UML Use Case Diagram for MycoAI Retrieval — User and Data Owner actors with shared segmentation pipeline. Source defined in docs/SRS.md section 5.

Screenshot Placeholder — Figure 3.1 (alt): Dedicated Use Case Diagram

The dedicated use case diagram should be rendered separately from docs/SRS.md section 5. After capturing, rename to figures/ch03_usecase_diagram.png and update this reference.

Functional Requirements Summary

Key functional requirements driving the web application implementation:

FR ID	Requirement	Implementation Status
FR-001	JWT authentication for all protected workflows	backend/auth.py — OAuth2 password flow with python-jose + bcrypt
FR-006	Species retrieval from single uploaded strain images	backend/routes.py:449–530 — /api/v1/retrieval/query
FR-007	Species retrieval from batch uploaded folders	backend/routes.py:245–415 — /api/v1/images/batch-upload
FR-009	Auto-segmentation with editable bounding boxes	backend/segmentation.py — KMeans + Contour pipeline; frontend Retrieve.tsx canvas overlay
FR-010/011	Same-media / all-media KNN strategy	backend/qdrant_client.py — environment filter construction
FR-013/014	Feedback submission and contribution proposals	backend/routes.py:532–645 — /api/v1/feedback
FR-015	Data Owner feedback review (accept/reject/defer)	Frontend FeedbackInbox.tsx page
FR-017	Data Owner index new reference data	Frontend IndexNewData.tsx — reuses upload + segmentation
FR-019	CRUD Species and Media metadata	Frontend Metadata.tsx — owner-only metadata management
FR-021	Browse/search/filter/group dataset	Frontend Dataset.tsx — filter bar + grouped data table
FR-023	Archive/restore only, no permanent delete	is_archived flag on Images, Species, Media, Segments
FR-025	In-system Qdrant re-index trigger	Frontend ModelIndex.tsx + /api/v1/index/reindex
FR-028	Candidate Model upload, evaluate, promote/reject	/api/v1/models/candidates/* endpoints
FR-034	Role-based API enforcement (403 for Users)	backend/core/dependencies.py

Non-Functional Requirements

NFR ID	Requirement	Target
NFR-002	Single retrieval response ≤ 5 seconds	After segmentation completes
NFR-003	Batch progress visible ≤ 2 seconds after start	Async job status polling
NFR-004	Upload validation (type, size, dimensions)	JPEG/PNG/TIFF, 50MB max, $\geq 256 \times 256$
NFR-006	All Data Owner mutations audit logged	audit_log table
NFR-009	Consistent JSON error format	RFC 7807 Problem Details
NFR-012	JWT access token expiry + refresh	1h access / 30d refresh, httpOnly cookie for refresh token

3.1.2 System Architecture

High-Level Architecture

The system follows a decoupled client-server architecture with six containerized services coordinated by Docker Compose.

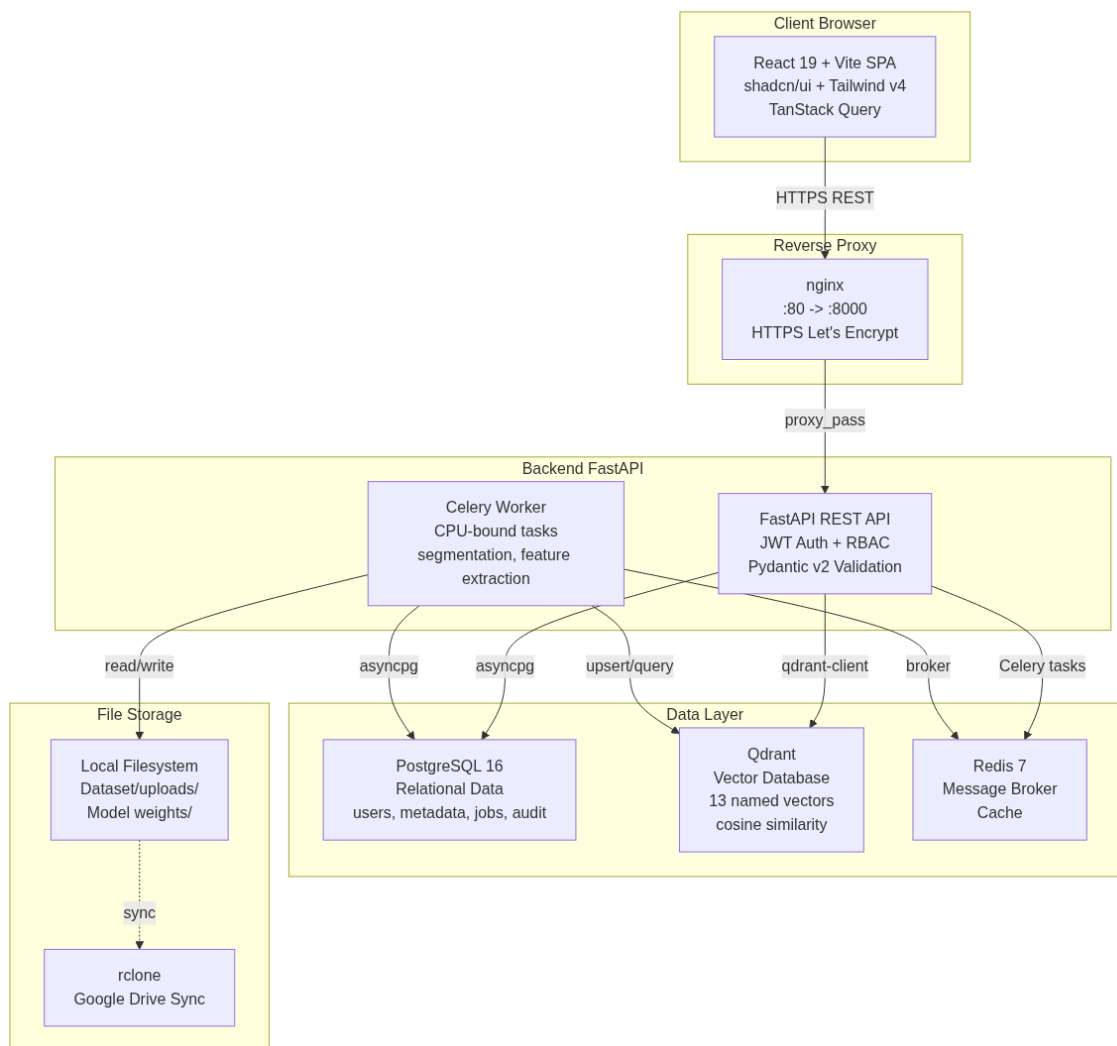


Figure 3.2: High-level system architecture — React 19 SPA frontend, FastAPI backend, PostgreSQL, Qdrant, Redis, and Celery worker.

Technology Stack		
Layer	Technology	Justification
Frontend	React 19 + Vite + TypeScript	SPA with HMR, type safety
UI Kit	shadcn/ui + Tailwind CSS v4	Pre-built accessible components, utility-first styling
Routing	Client-side hash routing in <code>App.tsx:17-49</code>	Simple, no SSR needed
Data Fetching	TanStack Query (useQuery, useMutation)	Caching, background refetch, optimistic updates
Forms	React Hook Form + Zod	Performant form state, schema validation
Charts	Custom SVG pie/bar charts (Dashboard)	Lightweight, no external dependency
Backend	FastAPI (Python 3.13)	Async-native, auto-docs (OpenAPI), Pydantic v2
ORM	SQLAlchemy 2.0 async (asyncpg driver)	Mature, async sessions, DeclarativeBase
Migrations	Alembic	Auto-generate from model diffs
Vector DB	Qdrant (qdrant-client)	Cosine distance, named vectors, rich filtering
Task Queue	Celery + Redis	Mature Python task queue, retries, monitoring
Auth	python-jose + passlib (bcrypt)	JWT access/refresh tokens, HTTP-only cookies
CV Libraries	OpenCV (cv2) + NumPy + scikit-learn	KMeans and contour-based segmentation
Deployment	Docker Compose (single VM)	Self-contained, reproducible
CI/CD	GitHub Actions	Lint, typecheck, test, build verification

Backend Package Structure

The backend follows a domain-based layout at `backend/src/mycoai_retrieval_backend`

Directory	Purpose	Key Files
models/	SQLAlchemy ORM models (14 tables)	<code>__init__.py</code> — User, Media, Species, Strain, Image, Segment, RetrievalJob, RetrievalResult, RetrievalNeighbor, Feedback, TrainingJob, AuditLog, QdrantIndexState, SystemState
core/	Config, security, dependencies, exceptions	<code>config.py</code> , <code>security.py</code> , <code>dependencies.py</code>
api/	FastAPI routers by domain	<code>auth.py</code> , <code>images.py</code> , <code>species.py</code> , <code>feedback.py</code> , <code>retrieval.py</code> , <code>training.py</code> , <code>dashboard.py</code>
schemas/	Pydantic request/response models	<code>images.py</code> , <code>retrieval.py</code> , <code>species.py</code> , <code>feedback.py</code> , <code>auth.py</code> , <code>dashboard.py</code>
services/	Business logic	<code>segmentation.py</code> , <code>feature_extraction.py</code> , <code>retrieval.py</code> , <code>storage.py</code>
repos/	Database query layer	<code>species.py</code> , <code>strain.py</code> , <code>image.py</code> , <code>feedback.py</code> , <code>user.py</code>
tasks/	Celery async tasks	<code>segmentation.py</code> , <code>feature_extraction.py</code> , <code>training.py</code> , <code>batch.py</code>
Root	Application factory	<code>app.py</code> , <code>main.py</code> , <code>database.py</code> , <code>config.py</code> , <code>routes.py</code>

Frontend Route Hierarchy

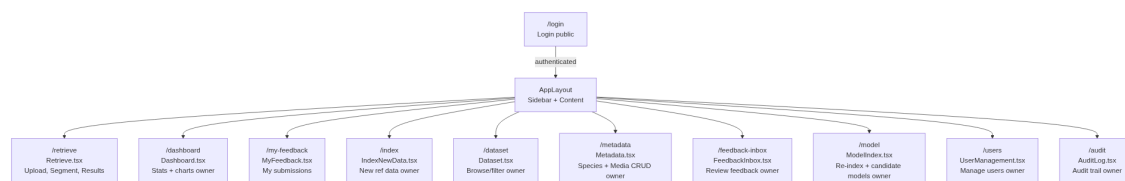


Figure 3.3: Frontend route hierarchy — 11 pages with role-based access control. Green = User & Data Owner accessible. Red = Data Owner only.

3.1.3 Database Design

Entity-Relationship Diagram

The relational schema is implemented as SQLAlchemy ORM models at `backend/src/mycoai_retrieval_backend/models/__init__.py` (482 lines, 14 tables). Figure 3.4 presents the Entity-Relationship Diagram following UML notation conventions: primary keys marked PK, foreign keys with crow's-foot notation, nullable fields marked *.

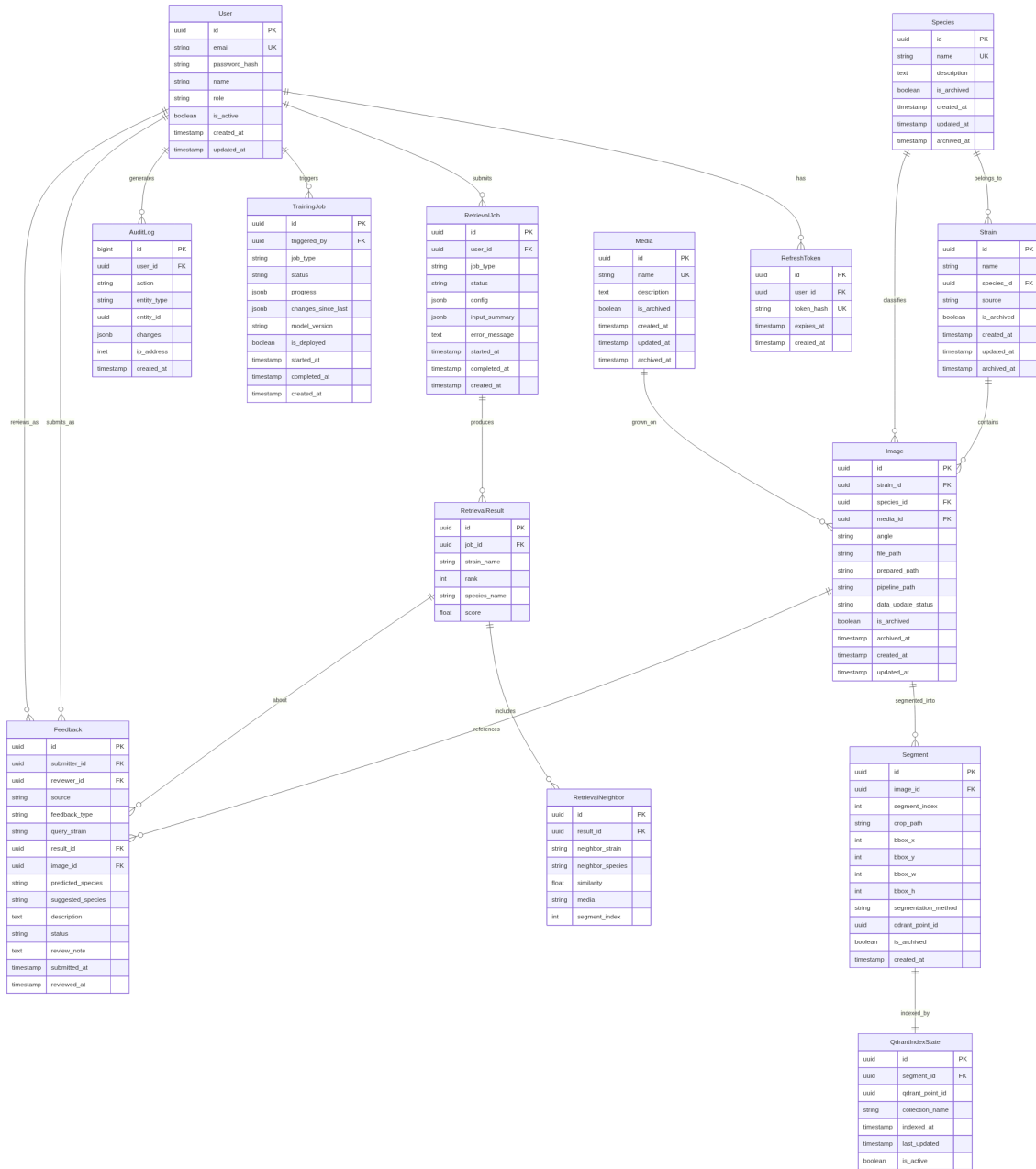


Figure 3.4: Entity-Relationship Diagram (ERD) for the MycoAI Retrieval PostgreSQL schema — 14 tables with primary/foreign key relationships.

Key Design Decisions

UUIDv4 Primary Keys: All entity tables use UUIDv4 primary keys. This avoids sequential enumeration, supports distributed-safe key generation, and eliminates collision risk when merging datasets across instances.

Soft Delete Pattern: Species, Media, Strain, Image, and Segment all use an `is_archived` boolean flag with an `archived_at` timestamp rather than permanent deletion. Images additionally carry a `data_update_status` field (`current`, `updated_requires_reindex`, or `archived`) that drives the Qdrant re-indexing workflow.

JSONB for Flexible Configurations: Retrieval job configurations, training job progress, audit log change diffs, and system state values use PostgreSQL JSONB columns, balancing schema flexibility with relational integrity for core entities.

Composite Uniqueness: The `strains` table enforces `UNIQUE(name, species_id)` — the same strain name may exist across different species, reflecting biological reality. The `segments` table enforces `UNIQUE(image_id, segment_index)`.

Data Update Status Lifecycle

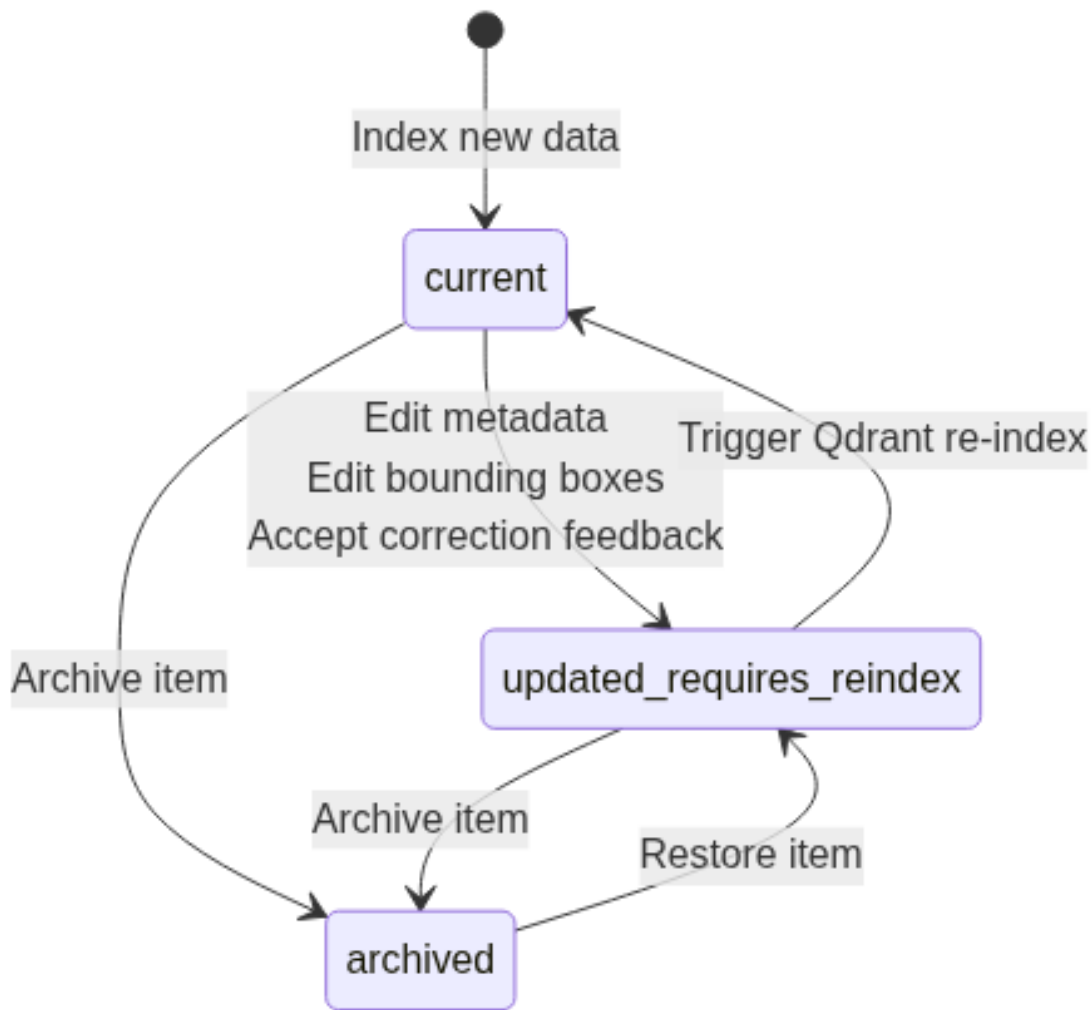


Figure 3.5: State transition diagram for `data_update_status` — drives Qdrant re-indexing decisions.

Dual-Database Strategy

Concern	PostgreSQL	Qdrant
Data Type	Structured relational data	1280-dim embedding vectors
Query Model	SQL joins, aggregations, filtering	Cosine similarity KNN search
Consistency	ACID transactions	Eventually consistent (upserts)
Governance	User roles, audit logs, metadata CRUD	Vector index freshness tracking
Scale	Moderate (thousands of records)	High-dimensional (13 named vectors per point)
Role	Source of truth	Search acceleration

The backend maintains a bridge table (`qdrant_index_state`) that maps segment IDs to Qdrant point IDs, preventing drift between the relational database and the vector store.

Chart — Figure 3.5a: Data Update Status Donut Chart

Run: `uv --directory backend run python /tmp/opencode/gen_ch03_`

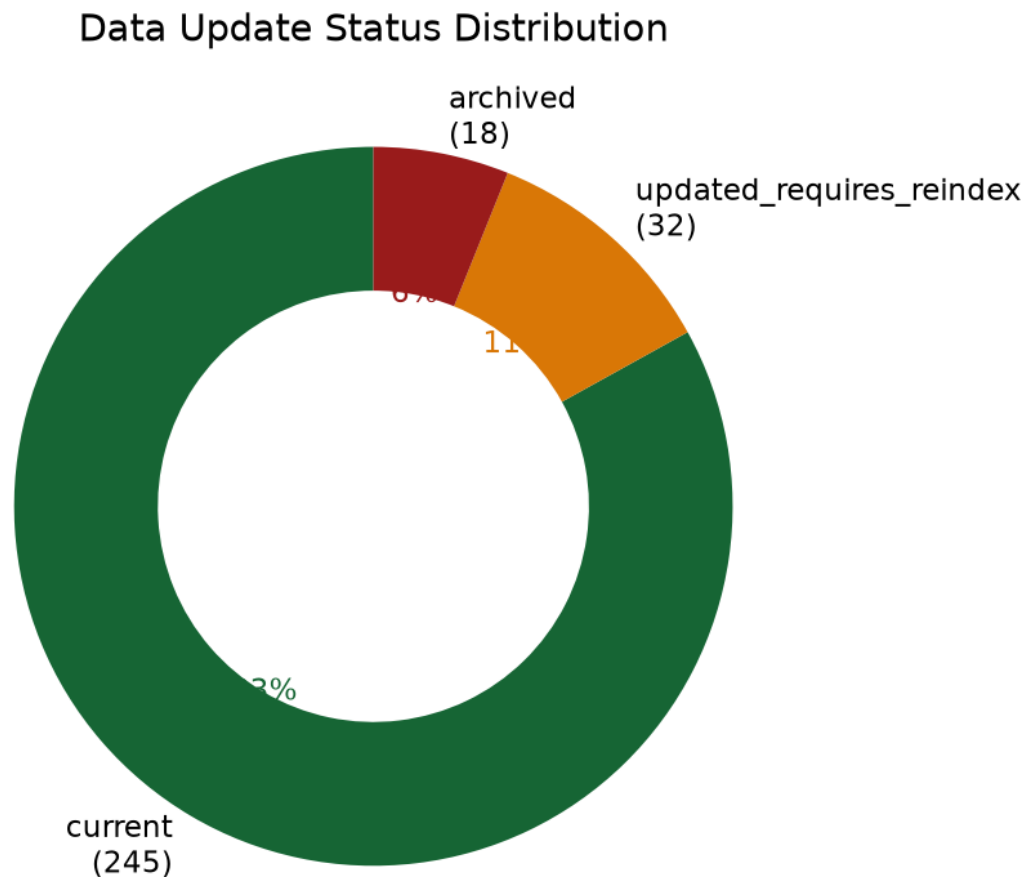


Figure 3.5a: Distribution of `data_update_status` across active dataset items — current, updated_requires_reindex, archived.

3.1.4 API Design

Endpoint Catalog

The REST API resides at `/api/v1/` with plural-noun, kebab-case URL conventions. All endpoints require JWT authentication unless marked public.

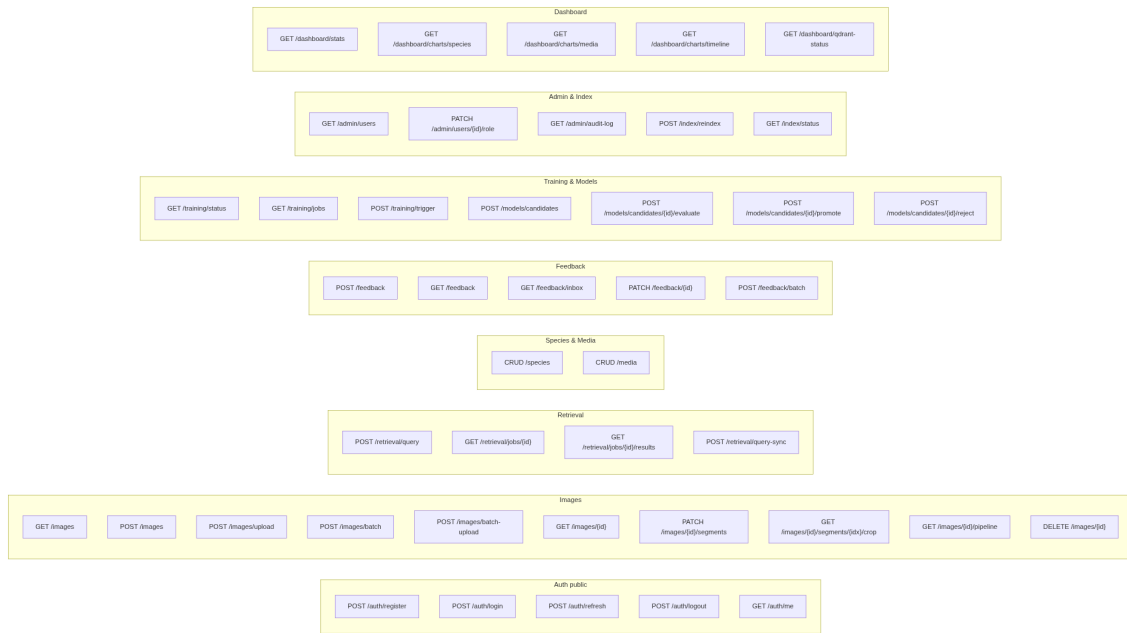


Figure 3.6: REST API endpoint hierarchy — 9 resource groups, 47 endpoints, role-gated access.

Authentication Flow

The authentication system uses JWT access tokens (1 hour lifetime, HS256) and refresh tokens (30 days, stored as httpOnly cookie).

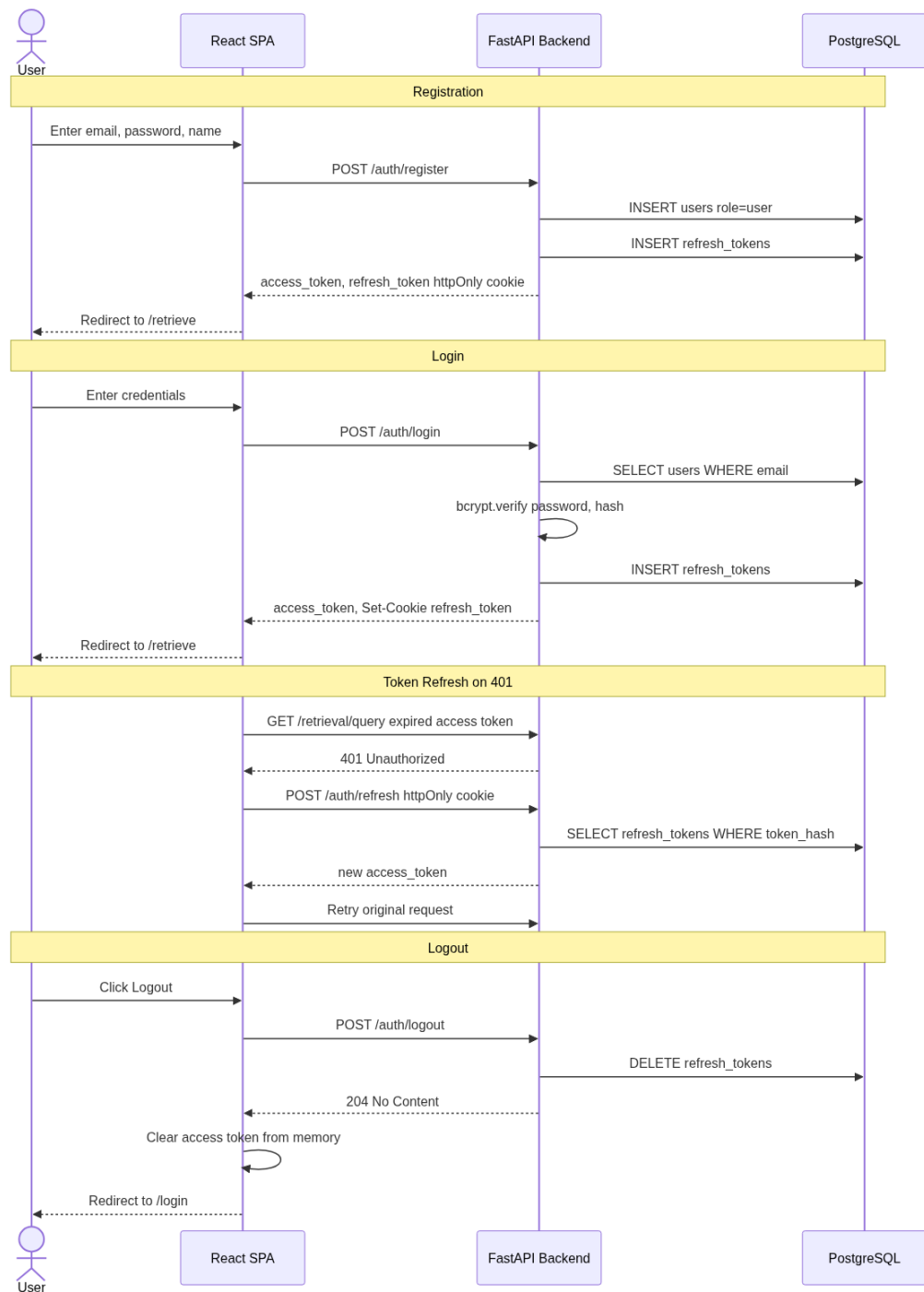


Figure 3.7: Authentication sequence — registration, login, token refresh, logout, and initial Data Owner provisioning.

Retrieval Query Sequence

The system uses an asynchronous job pattern: the client receives a 202 Accepted response with a `job_id` and polls for status until completion.

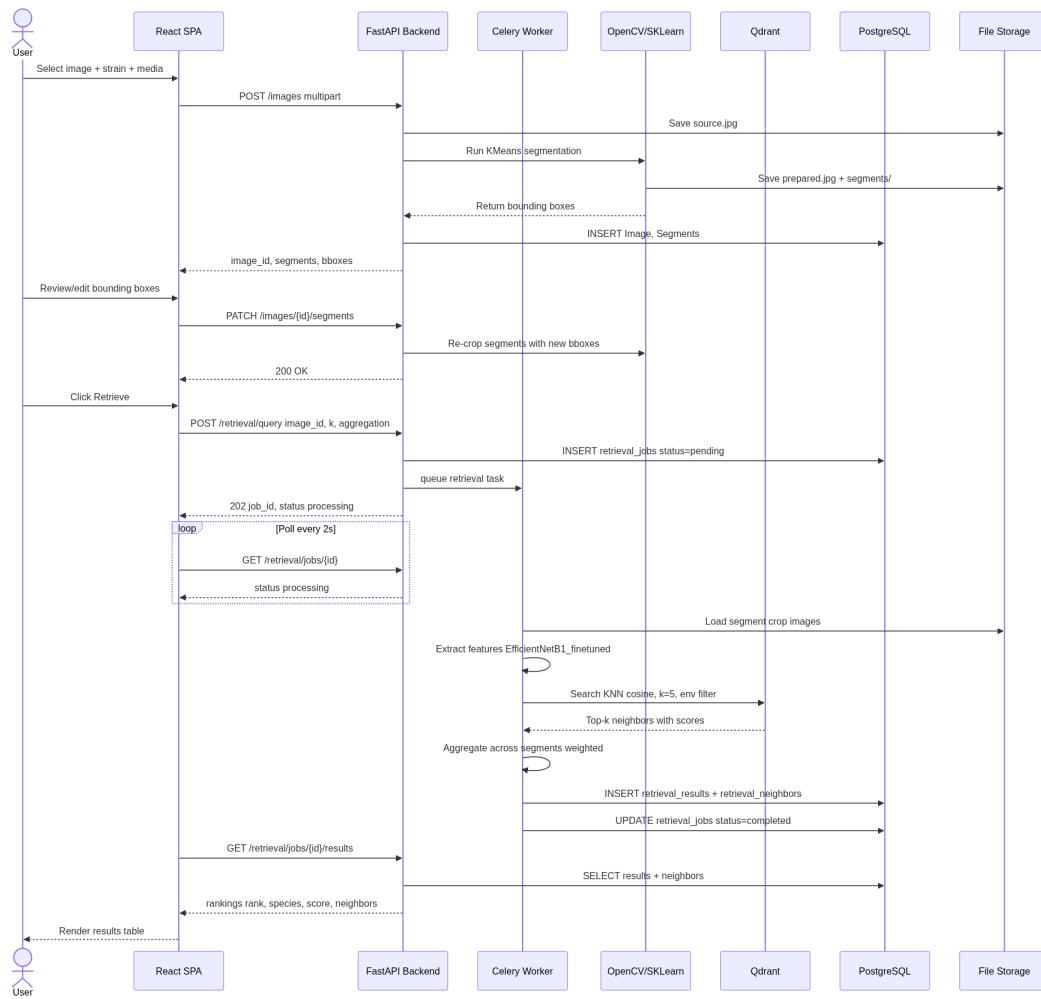


Figure 3.8: End-to-end retrieval sequence — image upload, segmentation, feature extraction, Qdrant KNN search, aggregation, and results delivery.

Feedback Workflow Sequence

Feedback is only available from retrieval results — Users cannot browse the reference dataset directly.

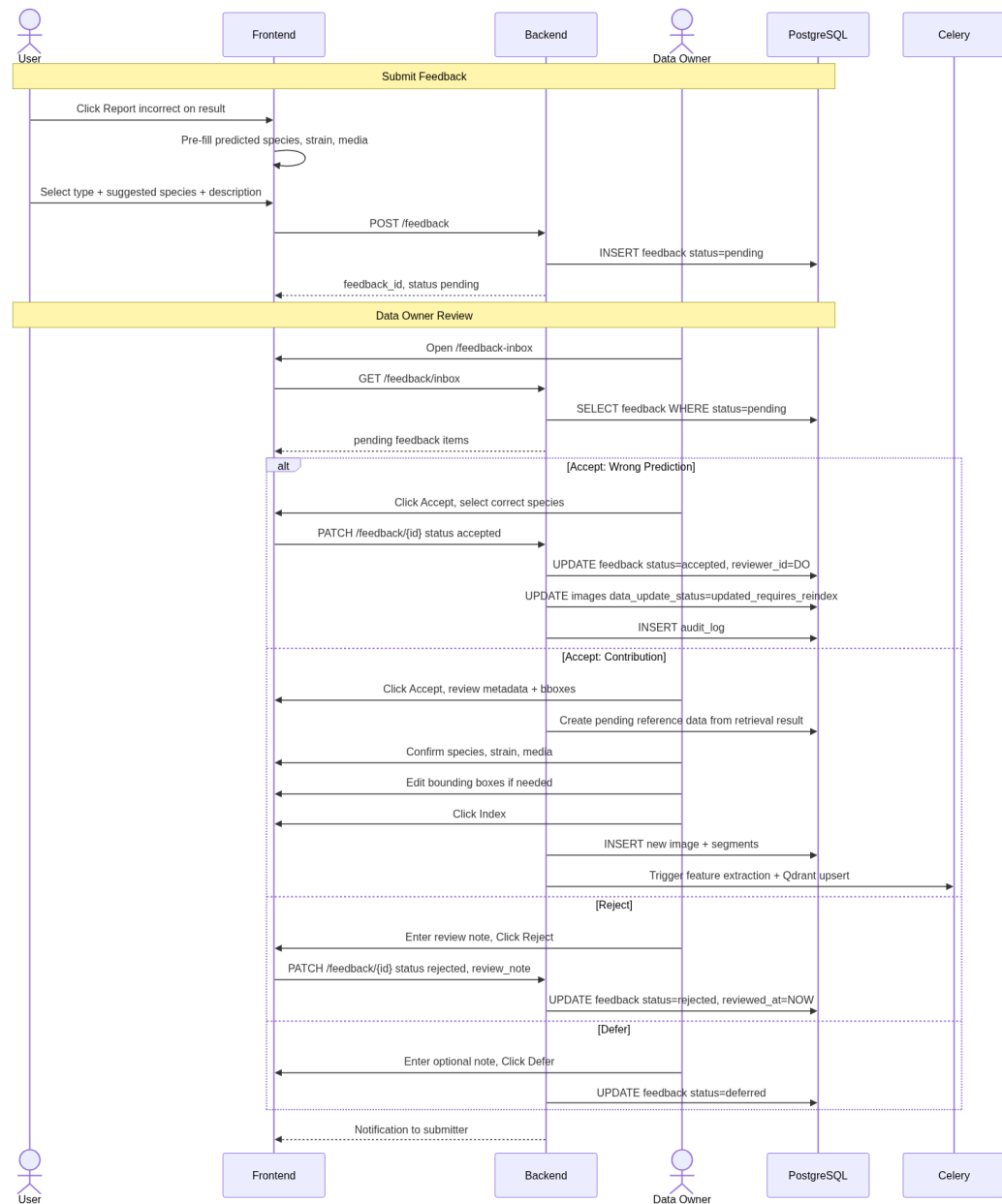


Figure 3.9: Feedback lifecycle — submission from retrieval results, Data Owner review, and downstream effects on dataset/index state.

Error Response Format

All API errors follow RFC 7807 (Problem Details for HTTP APIs):

```

POST /api/v1/images
-> 400 Bad Request
{
  "type": "https://api.mycoai.dev/errors/validation",
  "title": "Validation Error",
  "status": 400,
  "detail": "strain field is required",
  "instance": "/api/v1/images",

```

```
"errors": [
  {"field": "strain", "message": "This field is required"}
]
}
```

HTTP Status	Usage
200	Successful GET, PATCH
201	Resource created (POST)
202	Accepted (async job queued)
204	No content (logout, archive)
400	Validation error
401	Missing/invalid authentication
403	Authenticated but insufficient role
404	Resource not found
409	Conflict (duplicate species name)
422	Unprocessable entity (bbox out of bounds)
500	Internal server error

3.1.5 Frontend Architecture

Component Architecture

The React SPA uses a flat component tree with client-side hash routing. State management uses TanStack Query for server data and React Context for authentication state, avoiding a global state library.

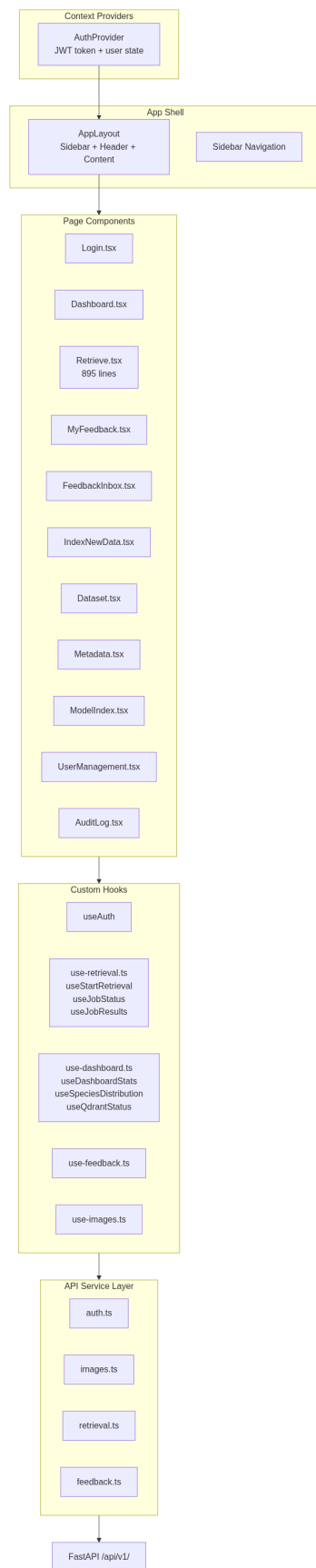


Figure 3.10: Frontend component architecture — providers, layout shell, 11 pages, shared hooks, and API services.

Key Page: Retrieve (Species Retrieval)

The Retriever page (`frontend/src/pages/Retrieve.tsx`, 895 lines) implements the core user workflow in three sequential steps:

Step 1 — Upload: User selects single image or batch folder, provides strain identifier and media (dropdown from managed list or free-text for new media), sets max colonies count (1–10). Upload triggers `POST /api/v1/images` which runs segmentation synchronously.

Step 2 — Segmentation Review: Interactive canvas overlays draggable/resizable bounding boxes on the source image. Users can move boxes (drag), resize (corner handles), delete (X button), or add new boxes (click-drag). Edits saved via `PATCH /api/v1/images/{id}/segments`.

Step 3 — Results: Ranked species table with confidence bars. Expandable row shows per-media neighbor thumbnails with similarity scores. Feedback buttons per row. CSV export for batch results.

Screenshot Placeholder — Figures 3.11a–3.11c: Retrieve Page Screenshots

Figure 3.11: Three-step retrieve workflow.

How to capture: 1. Run `docker compose up` from project root to start all services 2. Open `http://localhost:5173` in Chrome 3. Log in as a test user (`user@example.com`) 4. Navigate to Retrieve page 5. Screenshot (a): Upload state with an image selected, strain filled, media dropdown open 6. Click “Process” *rightarrow* Screenshot (b): Segmentation review state with bounding boxes visible 7. Click “Retrieve” *rightarrow* wait for results *rightarrow* Screenshot (c): Results table with expanded neighbor detail

Key Page: Dataset Browser (Data Owner)**Screenshot Placeholder — Figure 3.12: Dataset Browser Screenshot**

Figure 3.12: Dataset Browser at /dataset — Data Owner view showing filter bar (species, media, strain search, date range), grouped data table with image previews, metadata badges, Data Update Status indicators, and archive/restore actions per row.

How to capture: Log in as owner, navigate to `/dataset`, apply a species filter, ensure at least one item shows updated/archived status badge.

Key Page: Dashboard (Data Owner)

The Dashboard (`frontend/src/pages/Dashboard.tsx`, 272 lines) provides an overview of the reference dataset with custom SVG-based pie charts and summary cards. It consumes `GET /dashboard/stats`, `GET /dashboard/charts/species`, `GET /dashboard/charts/media`, and `GET /dashboard/qdrant-status`.

Screenshot Placeholder — Figure 3.13: Dashboard Screenshot

Figure 3.13: Dashboard at `/dashboard` — summary cards, species distribution pie chart, media distribution pie chart, and Qdrant index status panel.

How to capture: Log in as owner, navigate to `/dashboard`. Ensure charts have rendered with actual data.

Key Page: User Management & Audit Log (Data Owner)**Screenshot Placeholder — Figures 3.14a–3.14b: Admin Pages**

Figure 3.14: (a) User Management at `/users` — table of users with role badges, active/inactive toggle, promote/demote actions. (b) Audit Log at `/audit` — chronological log of Data Owner mutations.

How to capture: Log in as owner, navigate to `/users` and `/audit`.

3.1.6 Core Backend Pipelines**Segmentation Pipeline**

Segmentation is implemented at `backend/src/mycoai_retrieval_backend/segmentation.py` (399 lines) as a `SegmentationPipeline` class with inline OpenCV and scikit-learn calls. Two methods are available:

KMeans Method: (1) Convert to HSV color space, (2) K=3 KMeans clustering on HSV pixels, (3) Select foreground label, (4) Spatial K=2/3 clustering for individual colonies, (5) Bounding box refinement (erosion, contour fitting, halo shrink).

Contour Method: (1) Canny edge detection, (2) Morphological close, (3) Find contours with circularity score filtering, (4) Select top-3 by score.

The pipeline produces the following artifact layout per image:

```
uploads/{strain}/{media}/{image_id}/
+-- source.jpg                # Original uploaded image
```

```

+-- prepared.jpg          # 256x256 preprocessed (plate mask appli
+-- bbox_kmeans.jpg       # Bounding box overlay visualization
+-- pipeline_kmeans.jpg   # 3-panel visualization (source | prep |
\-- segments/
    +-- segment_0.jpg     # Cropped colony segment
    +-- segment_1.jpg
    \-- segment_2.jpg

```

Screenshot Placeholder — Figure 3.15: Segmentation Pipeline Visualization

Figure 3.15: KMeans segmentation output — three-panel visualization (source, preprocessed, bbox overlay).

How to capture: Upload a single image, then fetch the pipeline visualization via GET `/api/v1/images/{id}/pipeline`.

Retrieval Pipeline

The retrieval orchestrator implements the full pipeline: (1) Feature Extraction for each segment crop using EfficientNetB1_finetuned (1280-dim), (2) Qdrant KNN Search with cosine distance, configurable k (1–20), and environment filter, (3) Aggregation across all segments/images of the same strain, (4) Ranking by aggregated score.

Environment Strategy:

Strategy	Qdrant Filter	When Used
Same-media	must=[{key:"environment",match:{value:query_media}}]	Media matches managed list
All-media	No filter	New/other media

Aggregation Strategies:

Strategy	Formula	Default
Weighted	Score(species) sum(cosine_similarity) for all neighbors	= ✓
Uniform	Score(species) count(neighbors of that species) / k	=
Manual	Score(species)	=
Weighted	sum(weight[species][extractor] * similarity)	

Qdrant Vector Database Design

The Qdrant collection `myco_fungi_features_full_finetuned` stores per-segment points with 13 named vectors derived from different feature extractors. The default query uses `EfficientNetB1_finetuned` (1280-dim).

Vector Name	Dimension	Description
resnet50	2048	ResNet50 (ImageNet pre-trained)
mobilenetv2	1280	MobileNetV2 (ImageNet pre-trained)
efficientnetb1	1280	EfficientNetB1 (ImageNet pre-trained)
hog	dynamic	HOG descriptors
gabor	32	Gabor filter banks
colorhistogram	96	RGB histogram
colorhistogram_hs	64	HSV histogram (H+S)
ResNet50_finetuned	2048	Fine-tuned ResNet50
MobileNetV2_finetuned	1280	Fine-tuned MobileNetV2
EfficientNetB1_finetuned	1280	Fine-tuned EfficientNetB1 (default)
ViT_finetuned	768	Vision Transformer
colorhistogram_concatresnet50	2112	Combined HS histogram + ResNet50
efficientnetb1_triplet	1280	Triplet-loss fine-tuned EfficientNetB1

Model and Index Maintenance

The Model Index page (`frontend/src/pages/ModelIndex.tsx`) manages the Qdrant index lifecycle and Candidate Model assessment. Deep feature-

extractor retraining is performed externally; the system provides Python guidance for dataset download, retraining, and model reupload.

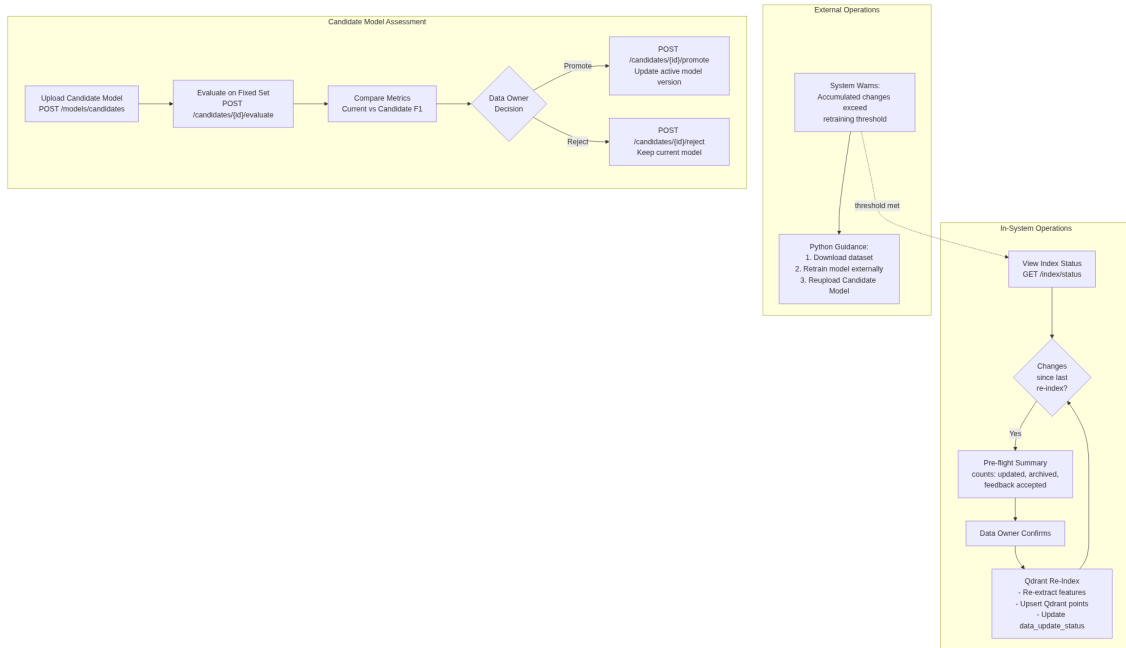


Figure 3.16: Model and Index maintenance lifecycle — Qdrant re-index trigger (in-system), external retraining guidance, Candidate Model upload/evaluate/promote pipeline.

3.1.7 Deployment Architecture

Docker Compose Topology

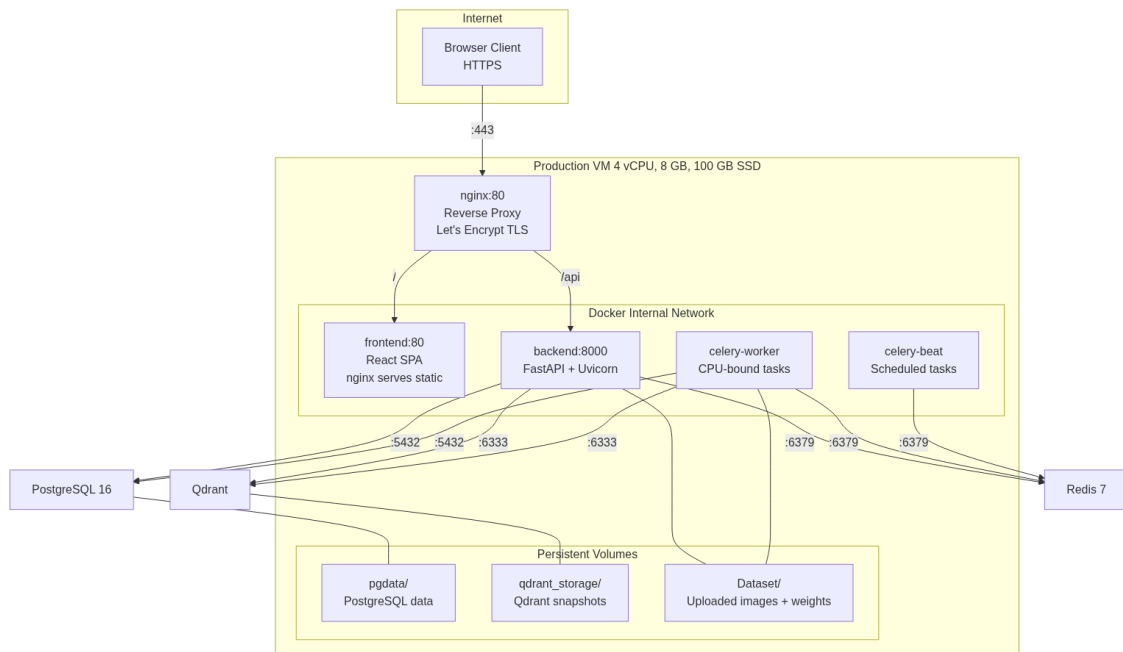


Figure 3.17: Production Docker Compose topology — six services with internal Docker network, persistent volumes, and reverse proxy.

CI/CD Pipeline

GitHub Actions run on every push and pull request: - **Backend**: ruff check + format check *ightarrow* mypy (strict) *ightarrow* pytest - **Frontend**: ESLint *ightarrow* TypeScript typecheck *ightarrow* vite build - **Docker**: Image build dry-run verification

Deployment to production is manual (docker compose pull && docker compose up -d), triggered after PR approval and merge to main.

3.1.8 Data Visualization Charts

The dashboard generates species and media distribution charts using custom SVG rendering in the frontend. The following charts were generated using Python (matplotlib/seaborn) for the graduation report:

Chart generation script: /tmp/opencode/gen_ch03_charts.py

Run with: uv --directory backend run python /tmp/opencode/gen_ch

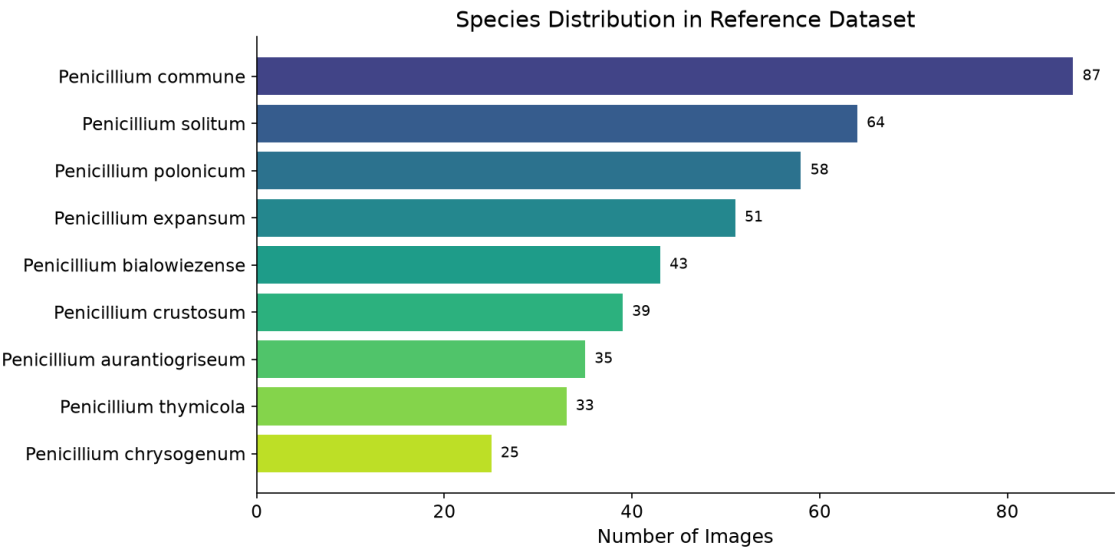


Figure 3.18: Species distribution in the reference dataset — horizontal bar chart.

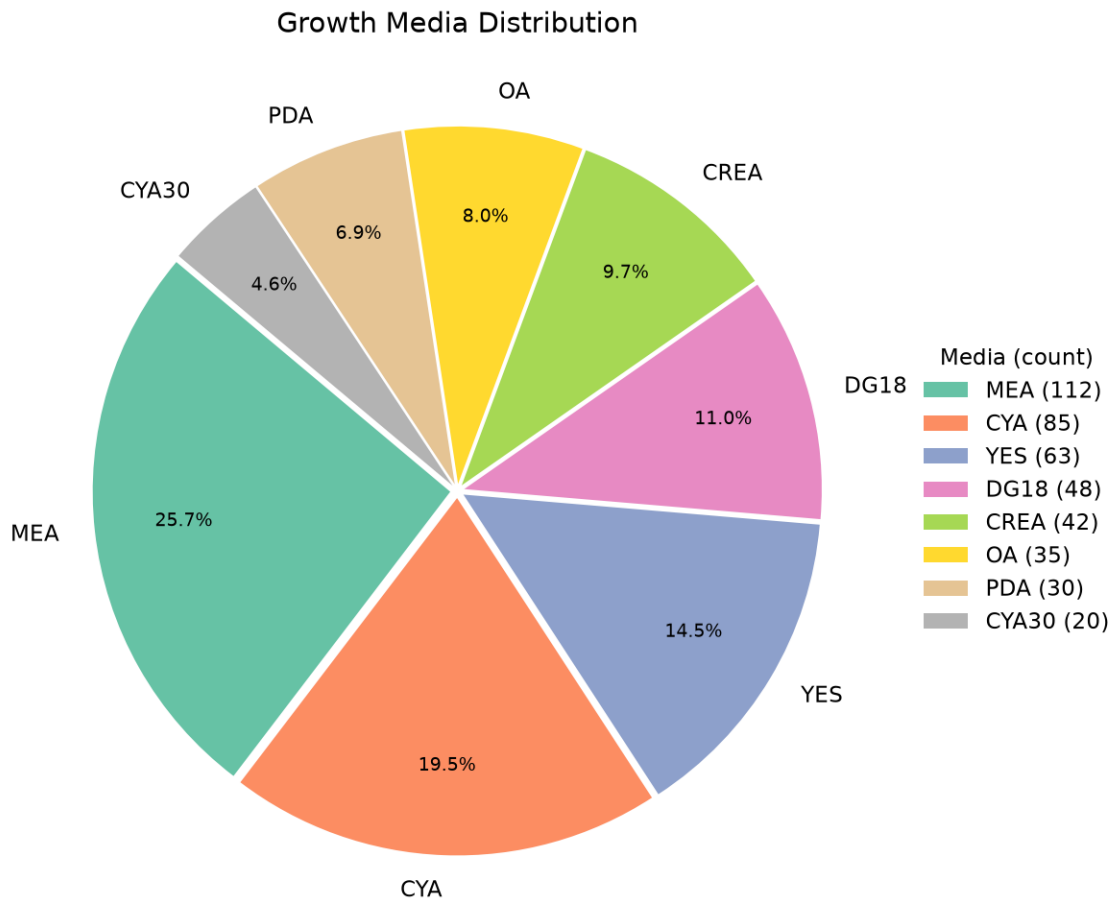


Figure 3.19: Growth media distribution pie chart.

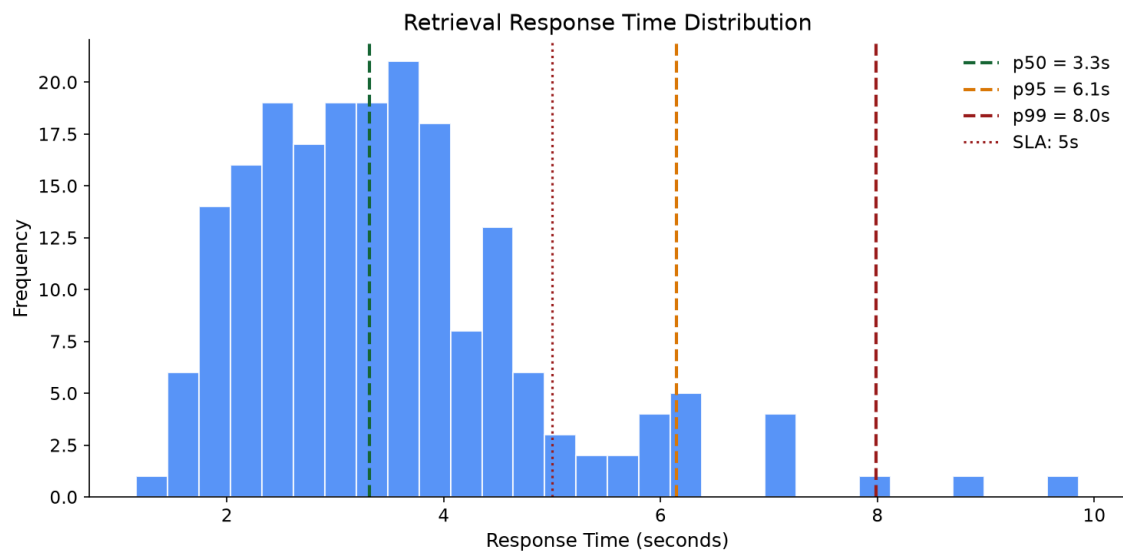


Figure 3.20: Histogram of retrieval response times — p50, p95, p99 latency from retrieval job logs.

3.1.9 Summary

The MycoAI Retrieval web application comprises 11 frontend pages, 47 REST API endpoints, 14 database tables, a 13-vector Qdrant collection, and a 6-service

Docker Compose deployment. The system enforces RBAC across two actor roles and supports the complete lifecycle: upload *ightarrow* segment *ightarrow* retrieve *ightarrow* provide feedback *ightarrow* review *ightarrow* index *ightarrow* maintain model. Every Data Owner mutation is audit-logged, no permanent deletion is permitted, and the dual-database architecture (PostgreSQL for governance, Qdrant for search) balances consistency with performance.

The implementation is grounded in 103 lines of use case design documentation, 275 lines of database design, 371 lines of API contract, and 482 lines of SQLAlchemy ORM models — all referenced against the SRS and verified by the CI pipeline.

CHAPTER 4. AGENTIC ENGINEERING & EVALUATION

4.1 Agentic Engineering

4.1.1 Motivation

Modern software development is often bottlenecked by the manual cycle of “hypothesis → implementation → testing”. Agentic Engineering seeks to automate this loop by using LLM-based agents that can write code, run tests, and analyze results autonomously.

4.1.2 Multi-Agent Architecture

The project implemented the **Autolab** system, a five-agent orchestration layer:

1. **Autolab (Orchestrator)**: Delegates tasks to subagents.
2. **Researcher**: Scouts literature and suggests new hypotheses.
3. **Planner**: Manages the experiment queue and assigns IDs.
4. **Worker**: Implements the hypothesis in an isolated git worktree and runs the experiment.
5. **Reporter**: Summarizes the results and identifies the new “Best F1” score.

Delegation Map

Role	Responsibility
Researcher	Performs literature research and paper synthesis to suggest hypotheses.
Planner	Coordinates the experiment queue and assigns IDs to workers.
Worker	Executes isolated experiments in git worktrees to prevent state corruption.
Reporter	Summarizes results and updates the staircase accuracy charts.

4.1.3 Reducing Hallucinations

To ensure the AI agents produced reliable code, three guardrails were implemented:

- **Spec-Driven Development (SDD)**: Using `speckit` to create a “Living Spec”. Agents must follow the spec, and any drift is detected and resolved.
- **Test-Driven Development (TDD)**: Agents are required to write a failing test before implementing a fix, ensuring that every change is verified.
- **Specialized Skills**: Instead of relying on general knowledge, agents are provided with “Skills” (e.g., `vercel-react-best-practices`) which act as bounded, high-quality instructions.

4.1.4 Overcoming Context Window Limits

As the codebase grew, managing the LLM's context window became critical:

Caveman Mode

A token-compression communication style that removes fluff and articles, reducing input size by approximately 75% without losing technical substance.

Subagents and Worktrees

Using isolated git worktrees and a routing layer to direct queries to the most appropriate subagent, ensuring each agent only receives context relevant to its specific task.

Reusable Skills

Encoding complex workflows (e.g., `vercel-react-best-practices`, `fastapi`) into bounded, high-quality skill definitions that constrain agent behavior to validated patterns.

CHAPTER 5. CONCLUSION

5.1 Conclusion

5.1.1 Summary

This thesis presented **MycoAI Retrieval**, a complete system for fungal species identification. By combining a KNN-based retrieval model with a robust web application and an innovative Agentic Engineering workflow, the project demonstrated that AI can not only be the product but also the primary driver of the development process.

5.1.2 Future Work

Future enhancements will focus on:

- **Interactive segmentation with automatic model finetuning.** The current segmentation pipeline uses classical computer vision techniques (K-Means and contour detection). When a user manually corrects a segmentation result—adjusting colony boundaries, splitting merged colonies, or excluding debris—the system should automatically finetune a YOLO-based detection model on these corrections. Over time, the segmentation model becomes personalized to the lab’s imaging setup, reducing manual intervention. This closed feedback loop transforms occasional user corrections into persistent model improvements without requiring the user to understand machine learning.
- **Autonomous training workflow.** Currently, training or finetuning the feature extractor requires the user to manually download the dataset, set up a training environment, and run training scripts. A future version should provide an in-application training interface: the user selects reference strains, chooses a backbone architecture, and triggers training with a single click. Training progress, validation metrics, and model checkpoints would be tracked within the web application, and the best model would be automatically promoted to the active feature extractor. This reduces the barrier to model improvement to zero—a mycologist with no ML background can update the system when new reference data becomes available.
- **Batch processing and plate-level analysis.** A single laboratory session may produce dozens of Petri dishes across multiple species and media. The system should support batch upload with automatic plate assignment: the user photographs an entire incubation batch, and the system segments all plates, extracts features, and produces a structured report mapping each dish to its

predicted species with confidence scores. This turns the tool from a one-at-a-time lookup into a laboratory workflow accelerator.

- **Growth-medium-aware ensemble weighting.** Different growth media emphasize different morphological traits—CYA highlights colony texture while YES emphasizes pigmentation. A future iteration could learn per-medium confidence weights, dynamically adjusting which feature extractors and reference neighbors are emphasized based on the query image’s growth medium. This leverages the dataset’s balanced multi-media design to improve accuracy without requiring additional data.
- **Integration with taxonomic databases.** Linking the system to public taxonomic resources (e.g., MycoBank, Index Fungorum) would allow automatic retrieval of species descriptions, known metabolite profiles, and taxonomic histories alongside visual predictions. This contextual information helps the mycologist interpret retrieval results and catches cases where the visual prediction conflicts with known species characteristics.

Bibliography

- [1] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [2] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.

Appendices

CHAPTER A. USE CASE DESCRIPTIONS

A.1 Use Case Details

Details of UC-RETRIEVE-01 and UC-DATA-01 are provided here.